

Environmental Statistics Book (Draft)

Lelys Bravo de Guenni and Kurt Ji

2025-08-17

Table of contents

Preface	4
1 Introduction	5
1.1 Why Environmental Statistics?	5
1.2 Most pressing Environmental problems of our times	5
2 Introduction to R	7
2.1 Getting Started With R	7
2.1.1 Environment Setup	7
2.1.2 Introduction to Packages	8
2.1.3 Variables	9
2.1.4 Numbers and Vectors	10
2.1.5 Strings and Boolean	10
2.1.6 Working with Dates	11
2.2 R Fundamentals	12
2.2.1 Math and Logical Operators	12
2.2.2 Functions	13
2.2.3 If/Else Statements	14
2.2.4 Loops	14
2.3 Data Fundamentals	15
2.3.1 Lists	15
2.3.2 Matrices and Arrays	16
2.3.3 Manipulating Matrices and Arrays	17
2.3.4 Dataframes	19
2.4 Read/Load/Output Data	20
2.4.1 Read Data	20
2.4.2 Write Data	20
2.4.3 Readr	20
2.5 Working with Dataframes	21
2.5.1 Manipulate Data	21
2.5.2 Apply functions	23
2.5.3 Dplyr	25
2.6 Plots and Graphics	27
2.6.1 Basic Plots	27
2.6.2 ggplot2	31

3	Exploratory Data Analysis, Probability Models and Basic Inference	36
3.1	Exploratory Data Analysis	36
3.2	Probability Models	38
3.2.1	Common Distributions and Sampling Using R	39
3.2.2	The Normal Model	47
3.2.3	The Central Limit Theorem	53
3.3	Statistical Inference	55
3.3.1	Introduction to Hypothesis Testing	55
3.3.2	Testing for Normality	61
3.3.3	Location Tests	62
3.3.4	Measures of Linear Association	66
4	Non-Parametric Methods for Statistical Inference	69
4.1	Introduction to One-Sample Rank Tests	69
4.1.1	Wilcoxon Signed-Rank Test	70
4.2	Rank Tests with Two or More Samples	75
4.2.1	Wilcoxon Rank-Sum test	76
4.2.2	Wilcoxon rank sum test and Mann-Whitney test	77
4.2.3	Kruskal-Wallis test	78
4.3	Non-parametric correlation measures	78
4.3.1	The Spearman's rank correlation	78
4.4	Kendal's rank correlation	79
4.5	Bootstrapp Confidence Intervals	79
4.5.1	Bootstrapping	79

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

1 Introduction

1.1 Why Environmental Statistics?

Statistics is a fundamental science in many applications. Normally we observe or collect data from an experiment, a phenomenon, an ecosystem, a population, and we use statistic for collecting, describing, visualizing, modelling and predicting important variables associated with the study problem.

The development of environmental statistics as a specific branch or section within statistics has dramatically grown in the past 25 years. The need for applications of a wide range of methods adapted to solve the complexities of environmental data collection, modeling and prediction has been recognized by many institutions worldwide. Many national and international statistical societies as the American Statistical Association (ASA), the International Statistical Institute (ISI), the Royal Statistical Society (RSS), the International Society of Bayesian Analysis (ISBA) and Statistical Society of Australia (SSA) have a devoted section to environmental statistics.

The application of Statistics to analyze environmental issues requires using a variety of tools and methodologies ranging from Multivariate Analysis, Time Series analysis to Spatial Statistics. Each application requires specific tools, and data exploration and visualization are necessary first steps to understand potential assumptions on probabilistic models to fit collected data.

1.2 Most pressing Environmental problems of our times

Environmental Statistics has a wide range of applications and offers a variety of tools to solve some of the most important environmental problems of our times:

- Biodiversity Loss: Many plant and animal species are rapidly disappearing due to the increment of human activities and environmental degradation. <https://www.europarl.europa.eu/news/en/headlines/society/20200109STO69929/biodiversity-loss-what-is-causing-it-and-why-is-it-a-concern>

- Air pollution, or the release of pollutants into the air, has negative impacts on human health. The World Health Organization (WHO) estimates that nine out of ten people breathe air with toxic pollutants exceeding the recommended thresholds by the WHO. <https://www.nrdc.org/stories/air-pollution-everything-you-need-know>
- Water pollution, or the release of chemicals or microorganisms into water reservoirs like streams, rivers, lakes, aquifers, oceans or any other water bodies, has become a public health issue worldwide. Water pollution sources from human activities include agriculture, sewage and wastewater, oil pollution/spills, radioactive waste. These sources of pollution would affect groundwater, surface water and oceans. <https://www.nrdc.org/stories/water-pollution-everything-you-need-know>
- Ecosystems degradation including habitat losses, loss of biodiversity and the spread of invasive species <https://www.sciencedirect.com/science/article/pii/B9780128227947000083> have important consequences on the availability of ecosystem services with harmful impacts on minorities and vulnerable populations. <https://www.nature.com/articles/s41467-021-23905-3>
- Climate change is one of the most pressing environmental problems of our times. According to the Inter-governmental Panel of Climate Change (IPCC) sixth report, “emissions of greenhouse gases from human activities are responsible for approximately 1.1°C of warming since 1850-1900”, and on average over the next 20 years, “global temperature is expected to reach or exceed 1.5°C of warming”. This degree of warming will cause increasing heat waves, shorter cold seasons and longer warm season. <https://www.ipcc.ch/2021/08/09/ar6-wg1-20210809-pr/>
- Deforestation is driven by agricultural expansion, increasing urbanization and illegal logging and mining around the world, and it has important consequences on biodiversity losses and climate change. <https://www.conservation.org/stories/11-deforestation-facts-you-need-to-know>

2 Introduction to R

2.1 Getting Started With R

R is an integrated suite of software facilities for data manipulation, calculation and graphical display. Among other things it has

- an effective data handling and storage facility,
- a suite of operators for calculations on arrays, in particular matrices,
- a large, coherent, integrated collection of intermediate tools for data analysis,
- graphical facilities for data analysis and display either directly at the computer or on hard copy, and a well developed, simple and effective programming language (called ‘S’) which includes conditionals,
- loops, user defined recursive functions and input and output facilities. (Indeed most of the system supplied functions are themselves written in the S language.)

The term “environment” is intended to characterize it as a fully planned and coherent system, rather than an incremental accretion of very specific and inflexible tools, as is frequently the case with other data analysis software. R is very much a vehicle for newly developing methods of interactive data analysis. It has developed rapidly, and has been extended by a large collection of packages. However, most programs written in R are essentially ephemeral, written for a single piece of data analysis.

2.1.1 Environment Setup

Before you start, you want to do these 3 things first:

- **Install R** R is a programming language
- **Install RStudio** RStudio is the most widely-used IDE for R
- **Install tidyverse** tidyverse is the collection of packages that are widely used for data analysis. The detailed introduction to tidyverse can be found in Section 1.2. You can use the below command in R to install tidyverse

```
# install.packages("tidyverse")
library(tidyverse)
library(dplyr)
library(magrittr)
```

- **The pipe operator**

The `%>%` operator, also known as the “pipe operator”, is a powerful feature introduced by the `magrittr` package in R. It allows for a more readable and concise way of chaining together multiple function calls or operations. The pipe operator facilitates the creation of efficient and streamlined data analysis pipelines.

```
# Using pipe operator to integrate commands
example <- data.frame(x = c(1, 2, 3, 4, 5), y = c(2, 4, 6, 8, 10))

# Calculate the mean of y for rows where x is greater than 2
mean_y <- example %>% subset(x > 2) %>% pull(y) %>% mean()

# Print the mean
print(mean_y)
```

2.1.2 Introduction to Packages

In R, “packages” are collections of functions, data, and documentation designed to extend the functionality of the base R system. These packages serve as valuable tools that provide specialized capabilities, making complex tasks easier to accomplish and expanding the scope of data analysis and statistical modeling. Each package is designed to address specific data analysis needs, making it easier for users to perform tasks from different disciplines.

Using packages in R is a seamless process. Once a package is installed, you can load it into your R session using the `library()` function. This makes all the functions and data sets within the package available for use. The packages and collections covered in this book are listed below. Most of them are integrated into the `tidyverse` collection.

- **MASS** The “MASS” package includes various functions for statistical modeling, linear and nonlinear regression, generalized linear models, time series analysis, clustering, and more. Additionally, it offers data sets used in the book’s examples and exercises, allowing users to replicate and experiment with the presented statistical methods.
- **Tidyverse** The `tidyverse` package is a collection of R packages that provides a consistent and coherent framework for data manipulation, exploration, and visualization, promoting a tidy data format and streamlined data analysis workflows. The core packages within the `tidyverse`, including `dplyr`, `ggplot2`, `tidyr`, `readr`, `lubridate`, `magrittr`

and **purrr**, among others, offer powerful tools for data import, transformation, visualization, and modeling.

- **magrittr** The **magrittr** package provides a set of operators, most notably the pipe operator `%>%`, that enhances code readability and promotes a more fluent programming style in R. By chaining functions together, **magrittr** allows users to create a more structured and expressive data manipulation workflow.
- **ggplot2** **ggplot2** is a versatile and comprehensive data visualization package based on the Grammar of Graphics. With **ggplot2**, users can effortlessly create sophisticated and aesthetically pleasing visualizations that effectively communicate insights from their data.
- **dplyr** **dplyr** is a powerful package designed for data manipulation tasks in R. With a concise syntax and intuitive functions like `filter()`, `arrange()`, `mutate()`, and `summarise()`, **dplyr** simplifies and streamlines data wrangling, making it an essential tool for data analysts and data scientists.
- **lubridate** **lubridate** is a package specifically tailored for working with dates and times in R. Its user-friendly functions enable users to parse, manipulate, and format date-time data, making time-related analysis and data manipulation tasks more efficient and accurate.
- **readr** **readr** is a fast and efficient package for reading and importing data into R. By providing easy-to-use functions to read various file formats, **readr** simplifies the data reading process and ensures accurate representation of data, enabling smoother data analysis workflows.

2.1.3 Variables

You can create variables like in all other programming languages using “`->{`” or “`=`”

```
x <- 21
print(x)
```

```
[1] 21
```

```
x <- "text"
print(x)
```

```
[1] "text"
```

2.1.4 Numbers and Vectors

You can also assign numeric values or vectors to your variables

- **Numbers can be in many forms**

```
# Create numbers in r
x <- 3.5 # numeric
y <- 3L # integer
z <- 1i # complex
```

- **Vectors**

```
# Using the combine operator `c()` in r
u <- c(1,1,2,3,5)
# Using the sequence operator `:` in r
v <- 1:5
```

```
print(u)
```

```
[1] 1 1 2 3 5
```

```
print(v)
```

```
[1] 1 2 3 4 5
```

```
# length of vectors
length(u)
```

```
[1] 5
```

2.1.5 Strings and Boolean

Other types of variables include strings and booleans

- **String**

```
# String
s <- "This is a string"
print(s)
```

```
[1] "This is a string"
```

- **Boolean**

```
# Boolean
a <- TRUE
b <- (1 > 2)
```

```
print(a)
```

```
[1] TRUE
```

```
print(b)
```

```
[1] FALSE
```

2.1.6 Working with Dates

Working with dates is supported by R. Date is a specific type of object implemented in R and supplemented by other packages such as lubridate

- Date formatting

```
# Date formatting
date <- as.Date("2023-07-10")
formatted_date <- format(date, "%A, %B %d, %Y")
print(formatted_date)
```

```
[1] "Monday, July 10, 2023"
```

```
# test
```

- Date manipulation

```
require(lubridate)
```

```
Loading required package: lubridate
```

```
Warning: package 'lubridate' was built under R version 4.5.2
```

```
Attaching package: 'lubridate'
```

The following objects are masked from 'package:base':

date, intersect, setdiff, union

```
# Adding days to a date
date <- as.Date("2023-07-10")
new_date <- date + 7
print(new_date)
```

```
[1] "2023-07-17"
```

```
# Subtracting months from a date
date <- as.Date("2023-07-10")
new_date <- date - months(3)
print(new_date)
```

```
[1] "2023-04-10"
```

```
# Calculating the difference between two dates
date1 <- as.Date("2023-07-10")
date2 <- as.Date("2023-06-10")
diff <- date1 - date2
print(diff)
```

Time difference of 30 days

2.2 R Fundamentals

To perform meaningful tasks, you need to use operators, functions and loops smartly

2.2.1 Math and Logical Operators

- Fundamental operators

```
# plus, minus, multiply and subtraction
x <- 1
y <- 2

# print multiple arguments together using `cat`
cat(x+y, x-y, x*y, x/y)
```

```
3 -1 2 0.5
```

- Logarithm and exponential

```
# print exp(2) and log(1)
cat(exp(y), log(x))
```

```
7.389056 0
```

- Operations on vectors

```
# compute the vector max,min and median
u <- c(-1, 0, 3, -12.5, 8.3)
cat(max(u), min(u), median(u))
```

```
8.3 -12.5 0
```

```
# compute the sum and length
cat(sum(u), length(u))
```

```
-2.2 5
```

- Logical operators

```
# and or
x <- TRUE
y <- FALSE

cat(x && y, x || y)
```

```
FALSE TRUE
```

2.2.2 Functions

Functions in R are blocks of organized code designed to perform specific tasks, allowing you to efficiently reuse and modularize your code.

```
# Write a function that calculates the correlation between 2 vectors of the same length
my_function <- function(v1, v2) {
  # add a guard clause for the input
  if (length(v1) != length(v2)) {
    stop("input vectors have different lengths")
  }

  mean1 <- mean(v1)
```

```

mean2 <- mean(v2)

numerator <- sum((v1 - mean1) * (v2 - mean2))
denominator <- sqrt(sum((v1 - mean1)^2) * sum((v2 - mean2)^2))

correlation <- numerator / denominator
return(correlation)
}

my_function(c(1,3,5,7), c(2,4,6,8)) # call the function named my_function

```

```
[1] 1
```

2.2.3 If/Else Statements

```

# Example: Determine if a number is even or odd
num <- 7

# use the mod operator `%%`
if (num %% 2 == 0) {
  result <- "Even"
} else {
  result <- "Odd"
}

# print text and variables together using `cat`
cat("The number", num, "is", result)

```

```
The number 7 is Odd
```

2.2.4 Loops

Loops in R, such as for and while, enable iterative execution of code for efficient repetitive tasks.

```

# Use a for loop to print the first 10 elements of the Fibonacci sequence
for (i in 1:10) {
  if (i == 1 || i == 2) {

```

```

    prevprev = 1
    prev = 1
    cat(1, " ")
  } else {
    curr = prevprev + prev
    cat(curr, " ")
    prevprev = prev
    prev = curr
  }
}

```

1 1 2 3 5 8 13 21 34 55

```

# Use a while loop to find all the divisors of 336
n = 336
divisor = 2

while (divisor <= sqrt(n)) {
  if (n %% divisor == 0) {
    cat(divisor, ' ') # Found a divisor, print it out
  }
  divisor <- divisor + 1
}

```

2 3 4 6 7 8 12 14 16

2.3 Data Fundamentals

2.3.1 Lists

We can store data in a list:

```

# List of strings
thislist <- list("apple", "banana", "cherry")

# Print the list
print(thislist)

```

```
[[1]]  
[1] "apple"
```

```
[[2]]  
[1] "banana"
```

```
[[3]]  
[1] "cherry"
```

```
# access a list element  
print(thislist[1])
```

```
[[1]]  
[1] "apple"
```

```
# access multiple elements  
print(thislist[1:2])
```

```
[[1]]  
[1] "apple"
```

```
[[2]]  
[1] "banana"
```

2.3.2 Matrices and Arrays

- Matrix

Matrix is 2-D

```
# Create a matrix  
mat <- matrix(c(1,2,3,4,5,6), nrow = 3, ncol = 2)  
  
# Print the matrix  
mat
```

```
      [,1] [,2]  
[1,]    1    4  
[2,]    2    5  
[3,]    3    6
```


- Array

Array can be more than 2-D

```
# create a 3-dimensional array
arr <- array(c(1,2,3,4,5,6,7,8), dim = c(2,2,2))

# Print the array
arr
```

, , 1

	[,1]	[,2]
[1,]	1	3
[2,]	2	4

, , 2

	[,1]	[,2]
[1,]	5	7
[2,]	6	8

2.3.3 Manipulating Matrices and Arrays

You can access, edit, do operations and even combine matrices and arrays using the parsing technique

- Access elements

```
# Print the 2nd row of a matrix
mat[2,]
```

[1] 2 5

```
# Use multi-level parsing
arr[1,2,1]
```

[1] 3

- Edit elements

```
# Change the second element of the first row
mat[1,2] = 96
# Change the second column of the first matrix
arr[,2,1] = c(97,98)
```

```
# Print the matrix
print(mat)
```

```
      [,1] [,2]
[1,]    1   96
[2,]    2    5
[3,]    3    6
```

```
print(arr)
```

```
, , 1
```

```
      [,1] [,2]
[1,]    1   97
[2,]    2   98
```

```
, , 2
```

```
      [,1] [,2]
[1,]    5    7
[2,]    6    8
```

- Perform element-wise operations

```
# Creating 2 matrices
mat1 <- matrix(1:4, nrow = 2)
mat2 <- matrix(5:8, nrow = 2)
```

```
# Addition of two matrices
print(mat1 + mat2)
```

```
      [,1] [,2]
[1,]    6   10
[2,]    8   12
```

```
# Element-wise multiplication of two matrices
print(mat1 * mat2)
```

```
      [,1] [,2]
[1,]    5   21
[2,]   12   32
```

- Combine matrices You can bind matrices use `rbind` and `cbind`

```
# Stack a matrix on the top of the other
print(rbind(mat1,mat2))
```

```
      [,1] [,2]
[1,]    1    3
[2,]    2    4
[3,]    5    7
[4,]    6    8
```

```
# Link matrices side by side
print(cbind(mat1,mat2))
```

```
      [,1] [,2] [,3] [,4]
[1,]    1    3    5    7
[2,]    2    4    6    8
```

2.3.4 Dataframes

Dataframe is a powerful object that allows you to perform all kinds of operations on your data

```
# create a dataframe
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)
```

```
# Print the data frame
Data_Frame
```

```
      Training Pulse Duration
1 Strength    100      60
2  Stamina    150      30
3   Other    120      45
```

```
# access dataframe elements using the dollar sign
print(Data_Frame$Pulse)
```

```
[1] 100 150 120
```

2.4 Read/Load/Output Data

Read data from local files, load data from online source, write data into local files.

2.4.1 Read Data

```
# Load text file
data <- read.csv("data/CMI_TRH.csv")

head(data, 5)
```

	year	month	day	avg_rel_hum	avg_air_temp_f	avg_air_temp_c	date_string
1	1989	2	16	87.7	26.6	-3.0000000	2/16/1989
2	1989	2	17	89.4	26.4	-3.1111111	2/17/1989
3	1989	2	18	88.0	27.1	-2.7222222	2/18/1989
4	1989	2	19	96.7	32.5	0.2777778	2/19/1989
5	1989	2	20	100.0	34.5	1.3888889	2/20/1989

	date
1	2/16/1989
2	2/17/1989
3	2/18/1989
4	2/19/1989
5	2/20/1989

2.4.2 Write Data

```
# write.csv(data, file = "filename.csv", row.names = FALSE)
```

2.4.3 Readr

The **read** package is designed to read rectangular data (like 'csv', 'tsv', and 'fwf'). You can use the **readr** package to perform more advanced file operations in **r**

```
# install.packages("readr")
library(readr)
```

Warning: package 'readr' was built under R version 4.5.2

- `read_csv()`: comma-separated values (CSV)
- `read_tsv()`: tab-separated values (TSV)
- `read_csv2()`: semicolon-separated values with , as the decimal mark
- `read_delim()`: delimited files (CSV and TSV are important special cases)
- `read_fwf()`: fixed-width files
- `read_table()`: whitespace-separated files
- `read_log()`: web log files

Operations of Readr

- You can specify column types with a series of codes, note we are using `read_csv` from the `readr` package, not the built-in `read.csv`

```
# Read with custom column types
data <- read_csv("data/CMI_TRH.csv",
                 col_types = cols(
                   year = col_integer(),
                   month = col_integer(),
                   day = col_integer(),
                   avg_rel_hum = col_double(),
                   avg_air_temp_f = col_double(),
                   avg_air_temp_c = col_double(),
                   date_string = col_character(),
                   date = col_date(format = "%m/%d/%Y") # be sure this fo
                 )
)
```

Now the “date” column in your dataset has the date type

2.5 Working with Dataframes

Dataframe is powerful with geostatistical data analysis.

2.5.1 Manipulate Data

- Parsing data

You can parse data by location to get different portions of the data.

```
# Print first 3 rows
data[1:3,]
```

```
# A tibble: 3 x 8
  year month   day avg_rel_hum avg_air_temp_f avg_air_temp_c date_string
<int> <int> <int>     <dbl>         <dbl>         <dbl> <chr>
1  1989     2    16      87.7           26.6           -3  2/16/1989
2  1989     2    17      89.4           26.4          -3.11 2/17/1989
3  1989     2    18       88           27.1          -2.72 2/18/1989
# i 1 more variable: date <date>

# Print first 3 rows and 3 columns
data[1:3,1:3]
```

```
# A tibble: 3 x 3
  year month   day
<int> <int> <int>
1  1989     2    16
2  1989     2    17
3  1989     2    18
```

- Add a new column

```
# Remember this dataset we used?
head(Data_Frame)
```

```
Training Pulse Duration
1 Strength    100      60
2 Stamina    150      30
3   Other    120      45
```

```
# Add a new column
New_df <- cbind(Data_Frame, NewCol = c('new1','new2','new3'))

# A new column is added to the end
New_df
```

```
Training Pulse Duration NewCol
1 Strength    100      60  new1
2 Stamina    150      30  new2
3   Other    120      45  new3
```

- Remove a column

```
# Remove a column using the dollar sign
New_df$NewCol <- NULL

# The new column is gone
New_df
```

	Training	Pulse	Duration
1	Strength	100	60
2	Stamina	150	30
3	Other	120	45

2.5.2 Apply functions

The apply function family in R provides powerful tools for efficiently working with arrays, lists, and data frames by applying custom functions across dimensions, simplifying complex operations and data transformations.

- apply

Applies a function to rows or columns of an array-like object.

```
# apply the mean and sd function to a dataframe to get the mean and standard deviation of
df <- matrix(rnorm(25), ncol = 5) # 100 random variables organized into 5 columns
print(df)
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	-0.03393683	-1.8188035	-1.7335467	0.07534321	-0.1112040
[2,]	-0.71486857	-0.5960396	-0.1856236	0.21435832	-0.4884796
[3,]	-0.55077838	-0.9289702	0.6568466	0.87660546	0.1899543
[4,]	0.74689686	-1.2621512	-1.4688694	0.16644874	1.1560440
[5,]	-0.29323632	1.2497636	0.5583991	0.56881169	-1.1724230

```
apply(df, 2, function(x) c(mean=mean(x), sd=sd(x)))
```

	[,1]	[,2]	[,3]	[,4]	[,5]
mean	-0.1691846	-0.6712402	-0.4345588	0.3803135	-0.08522169
sd	0.5736442	1.1649477	1.1176167	0.3347101	0.86048838

- lapply

Applies a function to each element of a list, maintaining a list structure.

```
# use lapply to find the column sums of the 2 dataframes
list_of_dfs <- list(data.frame(x = 1:3, y = 4:6), data.frame(x = 7:9, y = 10:12))
print(list_of_dfs)
```

```
[[1]]
  x y
1 1 4
2 2 5
3 3 6
```

```
[[2]]
  x y
1 7 10
2 8 11
3 9 12
```

```
lapply(list_of_dfs, function(df) colSums(df))
```

```
[[1]]
  x y
6 15
```

```
[[2]]
  x y
24 33
```

- `sapply`

Applies a function to each element of a list, aiming to streamline output.

```
# use sapply to find the cumulative sum along columns
df <- data.frame(a = c(1, 2, 3), b = c(4, 5, 6), c = c(7, 8, 9))
sapply(df, cumsum)
```

```
      a  b  c
[1,] 1  4  7
[2,] 3  9 15
[3,] 6 15 24
```

- `mapply`

Applies a function to corresponding elements from multiple vectors or lists.

```
# Use mapply to find if each "divisor" divides its corresponding "integer"
integers <- c(14, 196, 1321, 948, 125)
divisors <- c(2, 4, 7, 12, 3)
mapply(function(s, r) s %% r == 0, integers, divisors)
```

```
[1] TRUE TRUE FALSE TRUE FALSE
```


2.5.3 Dplyr

`dplyr` is a powerful package within the `tidyverse` that provides a set of intuitive and efficient functions for data manipulation, transformation, and summarization in R.

```
# Load dplyr library
library(dplyr)
```

Warning: package 'dplyr' was built under R version 4.5.2

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

2.5.3.1 Study Data

- Descriptive Statistics

You can get descriptive statistics for a dataframe using `dplyr` and the ‘pipe’ operator easily.

```
# Calculate descriptive statistics using dplyr
summary_df <- data %>%
  summarise(
    Mean_Hum = mean(avg_rel_hum),
    Median_Hum = median(avg_rel_hum),
    SD_Hum = sd(avg_rel_hum),
    Min_Hum = min(avg_rel_hum),
    Max_Hum = max(avg_rel_hum),
    Count = n()
  )

print(summary_df)
```

```
# A tibble: 1 x 6
  Mean_Hum Median_Hum SD_Hum Min_Hum Max_Hum Count
  <dbl>      <dbl> <dbl>   <dbl>   <dbl> <int>
1   76.0      76.7  12.5    31.8    104. 12007
```

- **Select and Filter data**

```
# Select average humidity and date
selected_data <- data %>%
  select(avg_rel_hum, date_string)

print(selected_data[1:5,])
```

```
# A tibble: 5 x 2
  avg_rel_hum date_string
  <dbl> <chr>
1    87.7 2/16/1989
2    89.4 2/17/1989
3    88   2/18/1989
4    96.7 2/19/1989
5   100   2/20/1989
```

```
# Filter rows based on the value of average humidity
filtered_data <- data %>%
  filter(avg_rel_hum > 100)

print(filtered_data)
```

```
# A tibble: 4 x 8
  year month   day avg_rel_hum avg_air_temp_f avg_air_temp_c date_string
  <int> <int> <int>      <dbl>      <dbl>      <dbl> <chr>
1  2012    12     7    104.        44.4        6.89 12/7/2012
2  2012    12     8    101.        41.3        5.17 12/8/2012
3  2012    12     9    103.        42.1        5.61 12/9/2012
4  2012    12    20    102.        39.5        4.17 12/20/2012
# i 1 more variable: date <date>
```

- **Sort data**

```
# Sorting the data frame by "year" column in ascending order
data_sorted <- data %>%
  arrange(avg_rel_hum)

print(data_sorted[1:5, ])
```

```
# A tibble: 5 x 8
  year month   day avg_rel_hum avg_air_temp_f avg_air_temp_c date_string
<int> <int> <int>     <dbl>         <dbl>         <dbl> <chr>
1  2012     5    14      31.8           67.8           19.9 5/14/2012
2  2012     4     9      31.9           56.7           13.7 4/9/2012
3  2017     3    11      33.4           27.8           -2.33 3/11/2017
4  2018     4    29      35.3           49.1            9.5 4/29/2018
5  2000     5    25      35.4           66.4           19.1 5/25/2000
# i 1 more variable: date <date>
```

- **Group by data**

You can also find the aggregate values by grouping data.

```
# Find the mean average humidity of each year
grouped_data <- data %>%
  group_by(year) %>%
  summarise(Avg_Hum = mean(avg_rel_hum))

print(grouped_data[1:5, ])
```

```
# A tibble: 5 x 2
  year Avg_Hum
<int>   <dbl>
1  1989   87.1
2  1990   84.9
3  1991   82.2
4  1992   79.7
5  1993   80.7
```

2.6 Plots and Graphics

2.6.1 Basic Plots

The built-in `plot` function in R allows you to create scatter plots, line charts and other common graphics for statistical analysis.

- **scatter plot**

```
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.5.2
```

```
Warning: package 'tibble' was built under R version 4.5.2
```

Warning: package 'tidyr' was built under R version 4.5.2

Warning: package 'purrr' was built under R version 4.5.2

Warning: package 'stringr' was built under R version 4.5.2

Warning: package 'forcats' was built under R version 4.5.2

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v forcats 1.0.1      v stringr 1.6.0
v ggplot2 4.0.2      v tibble  3.3.1
v purrr   1.2.1      v tidyr   1.3.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to
```

```
library(dplyr)
library(magrittr)
```

Warning: package 'magrittr' was built under R version 4.5.2

Attaching package: 'magrittr'

The following object is masked from 'package:purrr':

set_names

The following object is masked from 'package:tidyr':

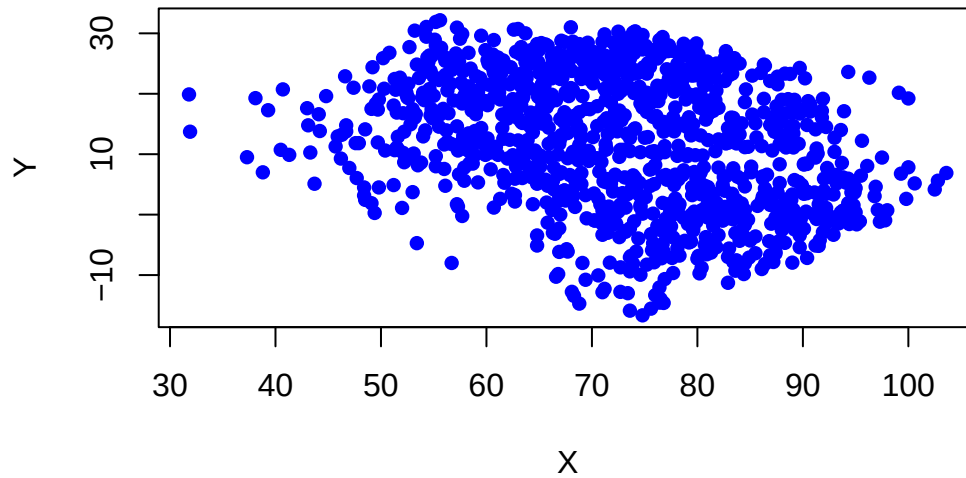
extract

```
# Select average humidity and date
data_between_2010_2012 <- data %>%
  filter(year >= 2010 & year <= 2012)
```

```
# Create a scatter plot
```

```
plot(data_between_2010_2012$avg_rel_hum, data_between_2010_2012$avg_air_temp_c, main = '')
```

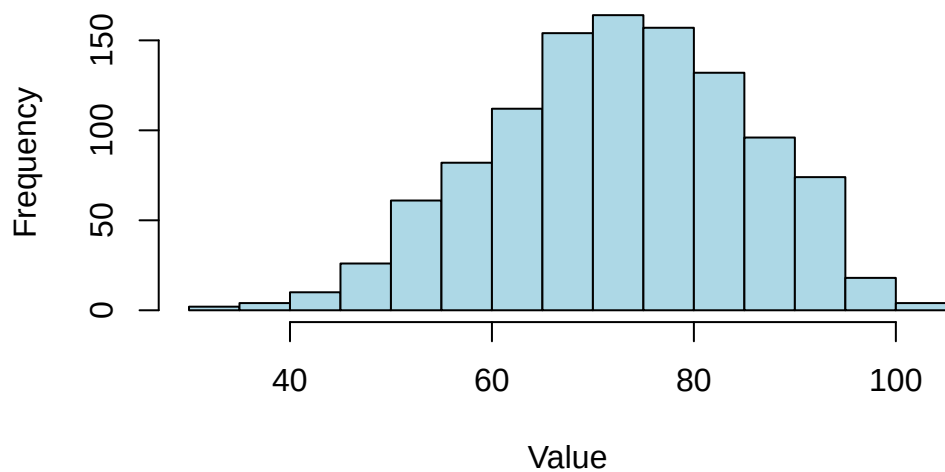
AVG Rel Humidity VS. AVG Rel Temperature in C, between 2010 and 2012



- histogram and bar plot

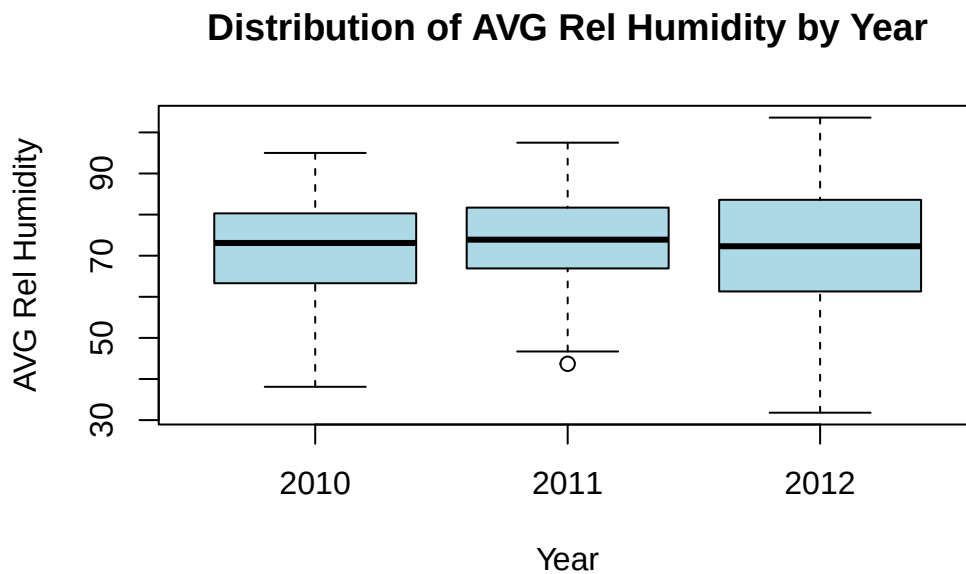
```
# Create a histogram  
hist(data_between_2010_2012$avg_rel_hum, main = "AVG Rel Humidity between 2010 and 2012")
```

AVG Rel Humidity between 2010 and 2012



- box plot

```
# Create a box plot for average realative humidity each year
boxplot(avg_rel_hum ~ year, data = data_between_2010_2012,
        main = "Distribution of AVG Rel Humidity by Year",
        xlab = "Year", ylab = "AVG Rel Humidity",
        col = "lightblue", border = "black")
```



- multiple plots and organize plots

You can use `par()` to organize the positions of multiple plots

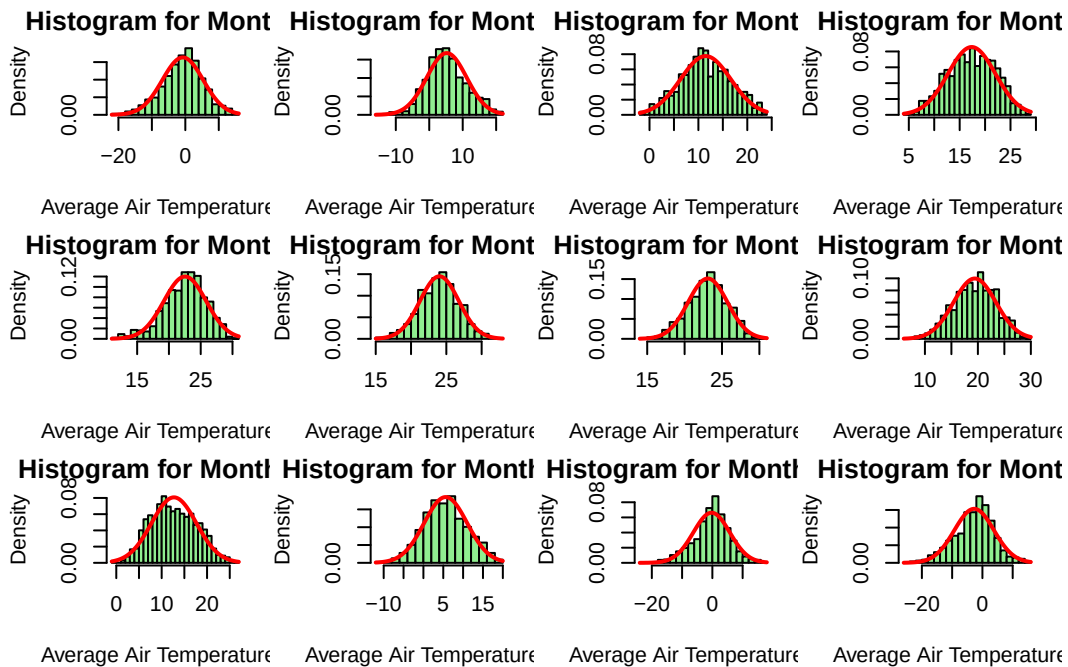
```
data_cmi <- read.csv("data/CMI_TRH.csv")

# Set up a 3x4 grid for 12 plots
par(mfrow = c(3, 4), mar = c(4, 4, 2, 1))

# Loop through each month
for (m in unique(data_cmi$month)) {
  # Subset data for the current month
  month_data <- subset(data_cmi, month == m)

  # Create a histogram
  hist(month_data$avg_air_temp_c, main = paste("Histogram for Month=", m), breaks=20,
        xlab = "Average Air Temperature [C]", col = "lightgreen", probability = TRUE)
```

```
# Add a bell curve
curve(dnorm(x, mean = mean(month_data$avg_air_temp_c), sd = sd(month_data$avg_air_
      add = TRUE, col = "red", lwd = 2)
}
```



```
# Reset par settings
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

2.6.2 ggplot2

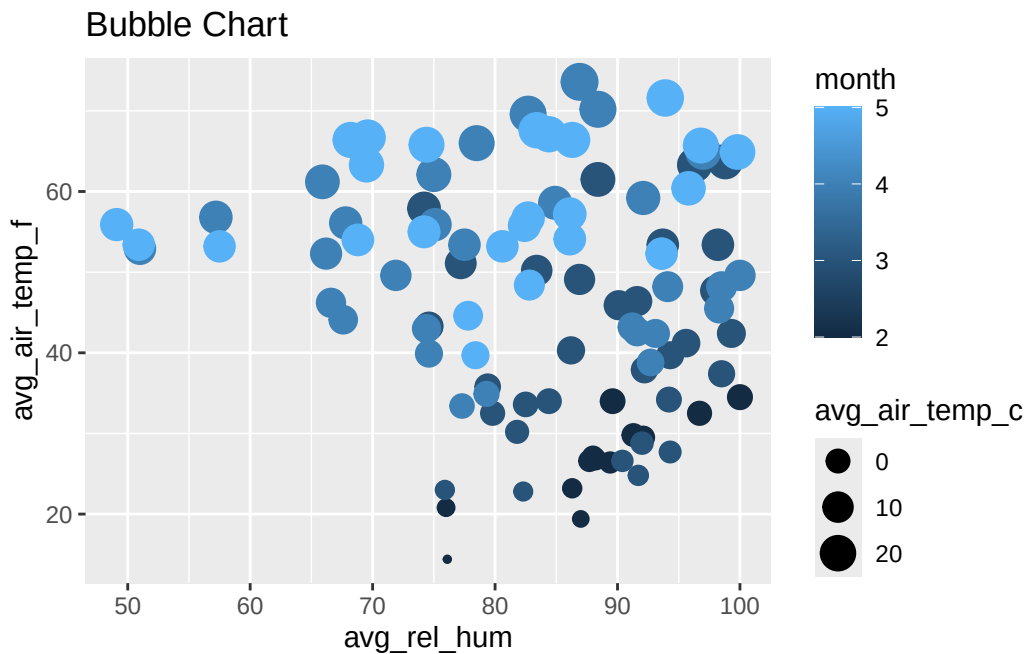
ggplot2 is another package that allows you to create elegant and customized plots with a concise and intuitive syntax. The **ggplot2** package uses a streamlined style of grammar with multiple “layers”, allowing you to compose your own plot in a more elegant and intuitive way.

```
# install.packages('ggplot2')
library(ggplot2)
```

- **Adding other dimensions for the plot**

You can also add color and size to represent more dimensions.

```
# Create the bubble chart
ggplot(data[1:100, ],
       aes(x = avg_rel_hum, y = avg_air_temp_f, size = avg_air_temp_c, color = month)) +
geom_point() +
labs(x = "avg_rel_hum", y = "avg_air_temp_f", title = "Bubble Chart")
```



- **Create insightful visualizations**

We will do an example to find the highest and lowest average air temperature in the year 2012, and plot them using `ggplot2`, along with the average relative humidity

```
# Find the highest temperature in 2012
data %>%
  filter(year(date) == 2012) %>%
  arrange(desc(avg_air_temp_f)) %>%
  head(1)
```

```
# A tibble: 1 x 8
  year month   day avg_rel_hum avg_air_temp_f avg_air_temp_c date_string
<int> <int> <int>     <dbl>         <dbl>         <dbl> <chr>
1  2012     7     7      55.6           89.9           32.2 7/7/2012
# i 1 more variable: date <date>
```

```
# Find the lowest temperature in 2012
data %>%
```



```

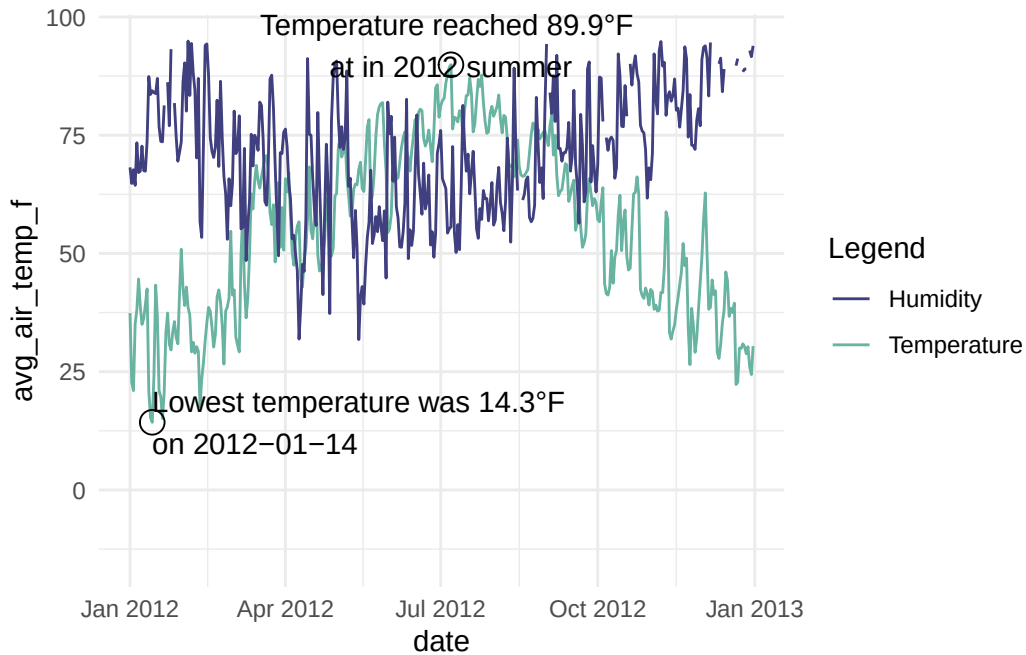
filter(year(date) == 2012) %>%
arrange(avg_air_temp_f) %>%
head(1)

# A tibble: 1 x 8
  year month   day avg_rel_hum avg_air_temp_f avg_air_temp_c date_string
  <int> <int> <int>      <dbl>          <dbl>          <dbl> <chr>
1  2012     1    14        84.4            14.3          -9.83 1/14/2012
# i 1 more variable: date <date>

data$date <- as.Date(data$date) # Convert date strings to date objects

# plot the temperature and humidity of the year 2012, labeling the highest and lowest po
data %>%
  filter(year(date) == 2012) %>% # Filter for the year 2012
  ggplot(aes(x = date)) +
    geom_line(aes(y = avg_air_temp_f, color = "Temperature"), linewidth = 0.5) +
    geom_line(aes(y = avg_rel_hum, color = "Humidity"), linewidth = 0.5) +
    ylim(-15, 95) + # Adjusted to include the range for -14.8
    annotate(geom = "text", x = as.Date("2012-07-07"), y = 93.9,
      label = "Temperature reached 89.9°F \nat in 2012 summer") +
    annotate(geom = "point", x = as.Date("2012-07-07"), y = 89.9, size = 4, shape = 21) +
    annotate(geom = "text", x = as.Date("2012-01-14"), y = 14.3, # Adjust y for visibil
      label = "Lowest temperature was 14.3°F \non 2012-01-14", hjust = 0) +
    annotate(geom = "point", x = as.Date("2012-01-14"), y = 14.3, size = 4, shape = 21) +
    scale_color_manual(values = c("Temperature" = "#69b3a2", "Humidity" = "#404080")) +
    labs(color = "Legend") +
    theme_minimal()

```

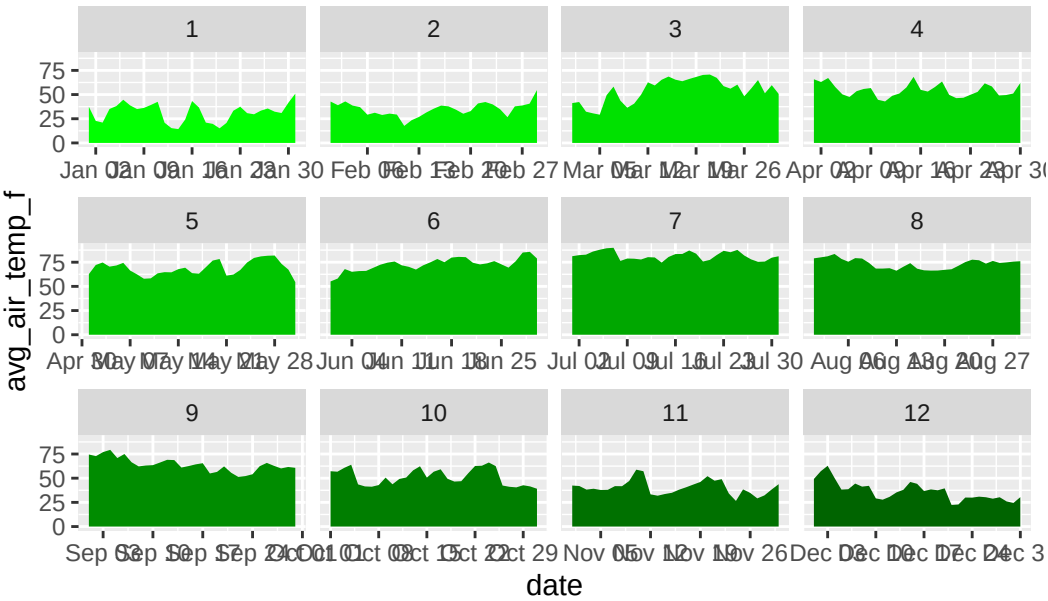


- **Plot multiple plots**

You can also plot multiple in one chart

```
data %>%
  filter(year(date) == 2012) %>%
  ggplot(aes(x=date, y=avg_air_temp_f, group=month(date), fill=month(date))) +
  geom_area() +
  ggtitle("Temperature by month in year 2012") +
  theme(
    legend.position="none",
  ) +
  facet_wrap(~month(date), scale="free_x") +
  scale_fill_gradient(low = "green", high = "darkgreen")
```

Temperature by month in year 2012



3 Exploratory Data Analysis, Probability Models and Basic Inference

3.1 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is a term proposed by the statistician John Tukey in the 1970's. The aim of EDA is to explore the data first, in order to understand the data structure and potential models by using graphical and quantitative tools.

Among the quantitative tools are the calculation of center tendency measures (mean, median and mode) and variability measures (variance, standard deviation, range, coefficient of variation, inter-quantile range).

Among the graphical tools are the graphics to propose potential probability models for the data set and detection of potential outliers (histograms and boxplots). An additional aim of EDA is to detect missing data. The researcher or data manager needs to take a decision on which methodology to apply to deal with missing data.

If the variable of interest is categorical, other graphical tools are more appropriate like the use of Barplots. Barplots will have the variable categories in the x axis, and the raw frequencies or relative frequencies in the y axis.

In a multivariate case, scatter-plots are very important for visualization of the relationships among the variables. Scatter-plots would help us to detect non-linearities among the variables.

```
library(ggplot2)
```

Warning: package 'ggplot2' was built under R version 4.5.2

```
# set seed
set.seed(123)

data_hw <- read.csv("data/hw_factors.csv")

# Selecting relevant columns
```

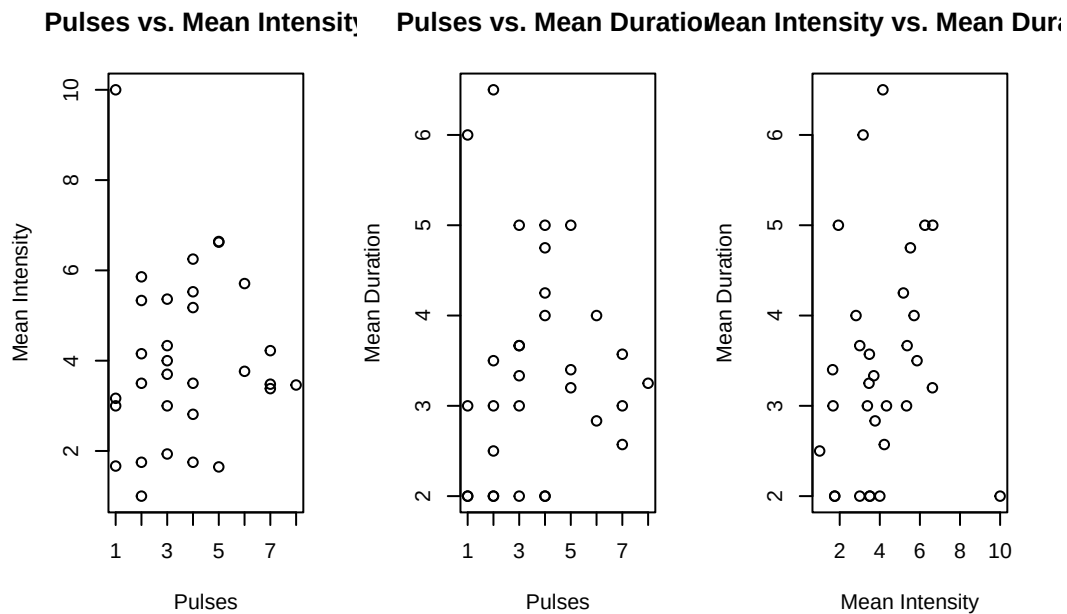
```
subset_data <- data_hw[, c("pulses", "mean_intensity", "mean_duration")]

# Creating scatter plots
par(mfrow = c(1, 3)) # Setting up a 1x3 layout

# Scatter plot 1: Pulses vs. Mean Intensity
plot(subset_data$pulses, subset_data$mean_intensity,
     xlab = "Pulses", ylab = "Mean Intensity", main = "Pulses vs. Mean Intensity")

# Scatter plot 2: Pulses vs. Mean Duration
plot(subset_data$pulses, subset_data$mean_duration,
     xlab = "Pulses", ylab = "Mean Duration", main = "Pulses vs. Mean Duration")

# Scatter plot 3: Mean Intensity vs. Mean Duration
plot(subset_data$mean_intensity, subset_data$mean_duration,
     xlab = "Mean Intensity", ylab = "Mean Duration", main = "Mean Intensity vs. Mean Duration")
```



Mean Air Temperature for Champaign

The Mean Air Temperature data set was discussed in Chapter 1.

We use this data set to perform an exploratory data analysis and perform some inference about the population parameters.

3.2 Probability Models

One of the main assumptions when we handle environmental variables is to deal with the issue of uncertainty. From the statistical point of view we will be assuming that many environmental variables are random variables. An immediate goal is to propose a probability model for the study variables.

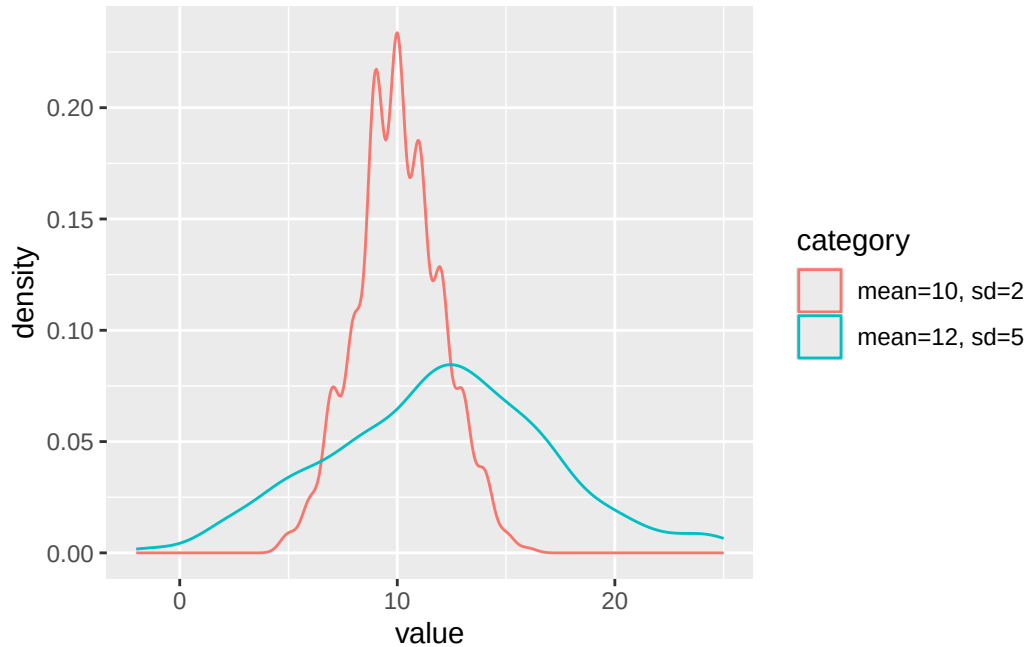
The normality assumption is a central assumption in most inference procedures. A first step in our analysis is plotting the data to understand their probability distribution.

The following example simulates two data sets from a normal distribution and plots the density function:

```
library(ggplot2)
# set seed
set.seed(123)

# create dataframe
df <- data.frame(category=factor(rep(c("mean=10, sd=2",
                                     "mean=12, sd=5"),
                                     each=500)),
                  value=round(c(rnorm(500, mean=10, sd=2),
                                rnorm(500, mean=12, sd=5))))

# Basic density plot with custom color
ggplot(df, aes(x=value, color=category)) +
# color property for changing color of plot
# geom_density() function plots the density plot
  geom_density()
```



3.2.1 Common Distributions and Sampling Using R

One of the main goals in the analysis of environmental data is the proposal of an appropriate statistical model for our data. This statistical model is usually specified by a set of unknown parameters which need to be estimated from the observations. Another important step in the analysis is to determine whether the proposed statistical model is correct.

The specification of a statistical model implies the proposal of a probability distribution for the variable of interest. Common probability models can be used for simulation, once the unknown parameters are estimated.

You can generate samples from different probability models in R using the `r_distribution` operation. You can specify the distribution parameters and the number of samples to be generated.

- **Normal Distribution**

The normal distribution, also known as the Gaussian distribution, is a symmetric bell-shaped curve commonly used to model continuous data with mean (μ) and standard deviation (σ) parameters. A random variable X has a Normal distribution if its probability density function (pdf) is of the form:

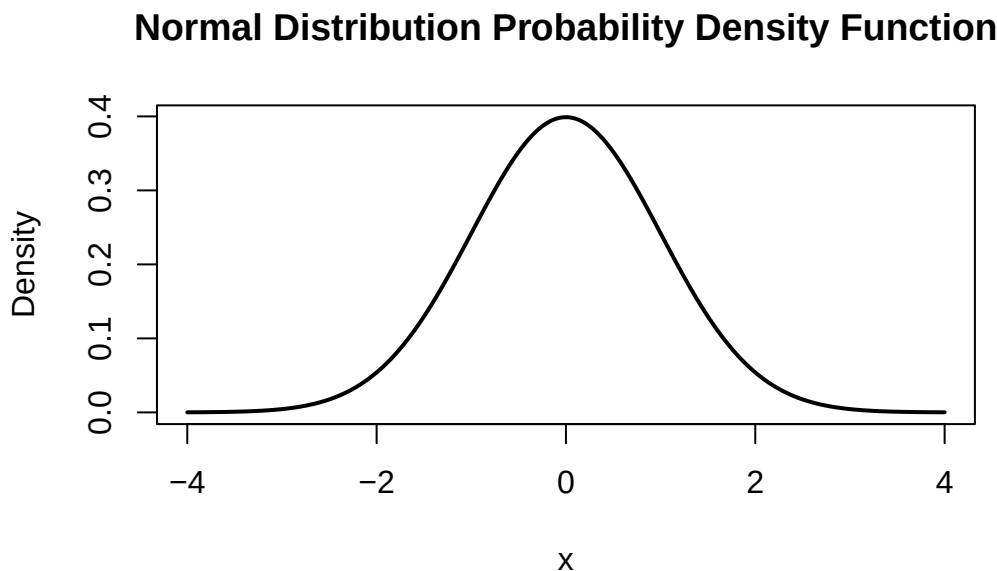
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where $\mu = E[X]$ and $\sigma^2 = Var[X]$ are the mean and variance of the pdf, and the random variable X can take values in the interval $(-\infty, \infty)$.

μ and σ^2 are also called the location and dispersion parameters of the Normal distribution.

```
# Generate density values for a normal distribution
x <- seq(-4, 4, length.out = 1000) # Specify the range of x values
density <- dnorm(x, mean = 0, sd = 1) # Mean = 0, Standard Deviation = 1

# Plot the normal bell curve
plot(x, density, type = "l", lwd = 2, col = "black",
      xlab = "x", ylab = "Density", main = "Normal Distribution Probability Density Function")
```



- You can sample from any known distribution using the `r+distribution()` function in R:

```
# Sample 250 values from a normal distribution with mean=5 and standard deviation of 2
norm_a <- rnorm(n = 500, mean = 5, sd = 2)

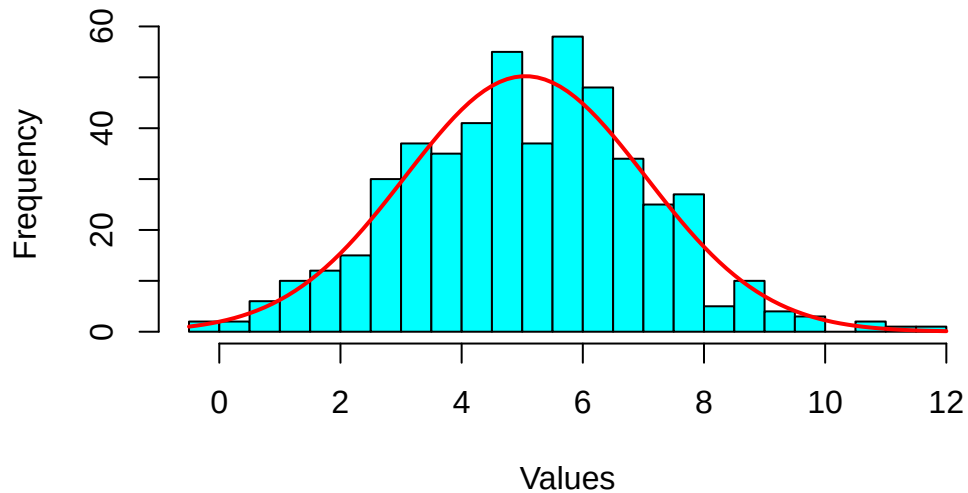
# Create a histogram without plotting it
hist_data <- hist(norm_a, breaks = 20, plot = FALSE)

# Create a histogram
hist(norm_a, breaks = 20, col = "cyan", main = "Sampled Data with a Normal Curve", xlab
```



```
# Add a normalized normal curve
curve(dnorm(x, mean = mean(norm_a), sd = sd(norm_a)) * diff(hist_data$breaks)[1] * length(norm_a),
      add = TRUE, col = "red", lwd = 2)
```

Sampled Data with a Normal Curve



```
# Set the width and height of the plot
options(repr.plot.width = 6, repr.plot.height = 4)
```

The symmetric bell curve can be observed from the histogram, which is symmetric about μ .

You can also plot the density curves using ggplot for sampled points using the function `geom_density`:

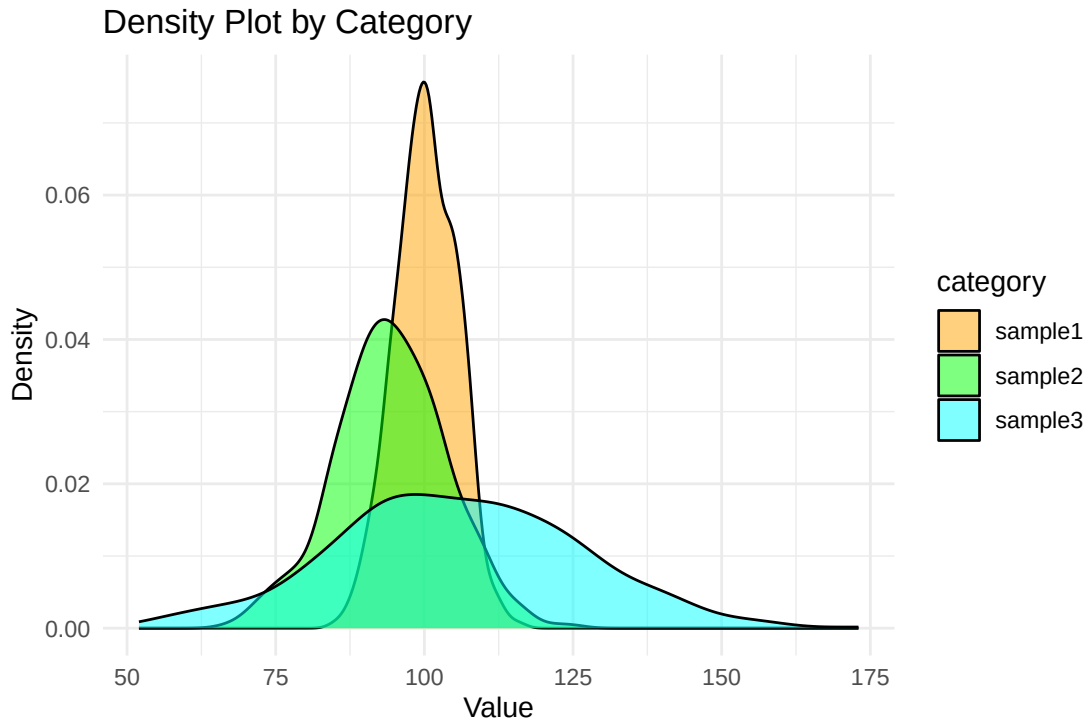
```
# Create a data frame of several normal variables with different parameters
df <- data.frame(category = factor(rep(c("sample1", "sample2", "sample3"), each = 500)),
                  value = round(c(rnorm(500, mean = 100, sd = 5),
                                rnorm(500, mean = 95, sd = 10),
                                rnorm(500, mean = 105, sd = 20))))

# Remember to load the library
library(ggplot2)

# Customize density plot
ggplot(df, aes(x = value, fill = category)) +
  geom_density(alpha = 0.5, color = "black", linewidth = 0.5) +
```

```
scale_fill_manual(values = c("sample1" = "orange", "sample2" = "green", "sample3" = "cyan"),
  labs(x = "Value", y = "Density", title = "Density Plot by Category") +
  theme_minimal()
```

Warning in geom_density(alpha = 0.5, color = "black", linewidth = 0.5): Ignoring unknown parameters: `linewidth`

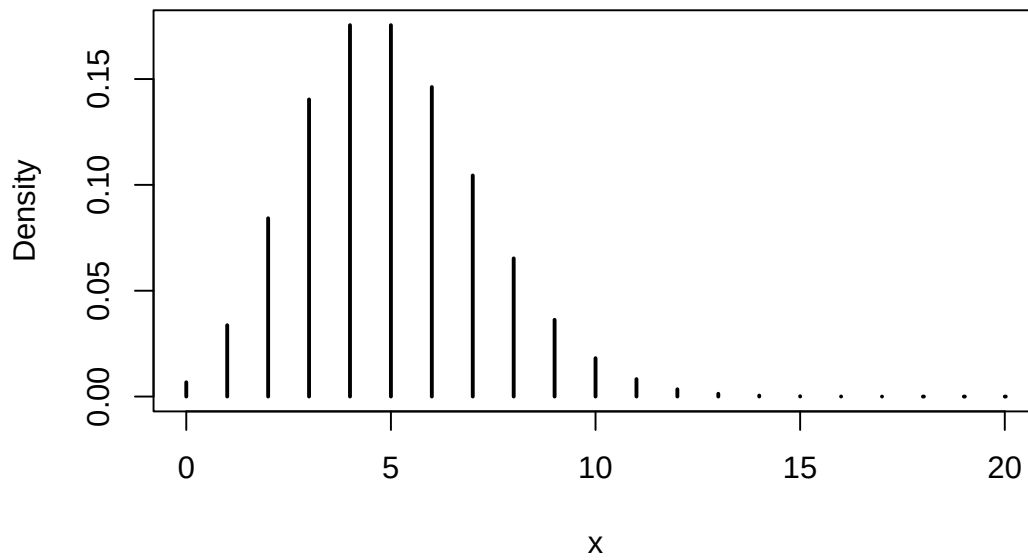


- **Poisson Distribution**

The Poisson distribution describes the number of events occurring in a fixed interval of time or space, or a given area. It is often used for counting rare events with a known average rate λ . Note that a poisson random variable can only take positive integer values.

$$P(X = k) = \frac{e^{-\lambda} \cdot \lambda^k}{k!}, k = 0, 1, 2, \dots$$

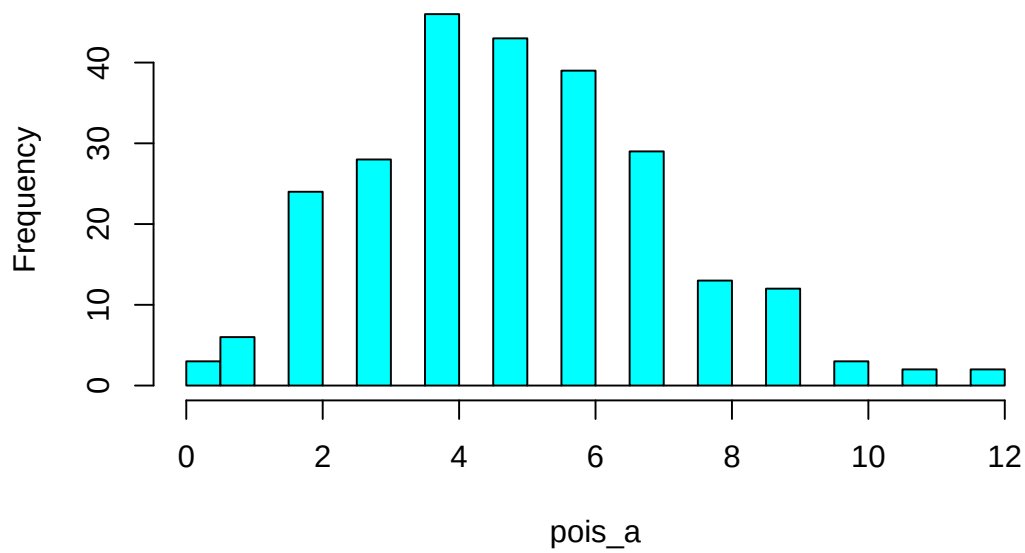
Poisson Probability Density Function



You can sample from Poisson distribution using the function `rpois`:

```
pois_a <- rpois(n = 250, lambda = 5)
hist(pois_a, breaks = 20, col = "cyan", main = "Density Plot of Sampled Poisson Data")
```

Density Plot of Sampled Poisson Data



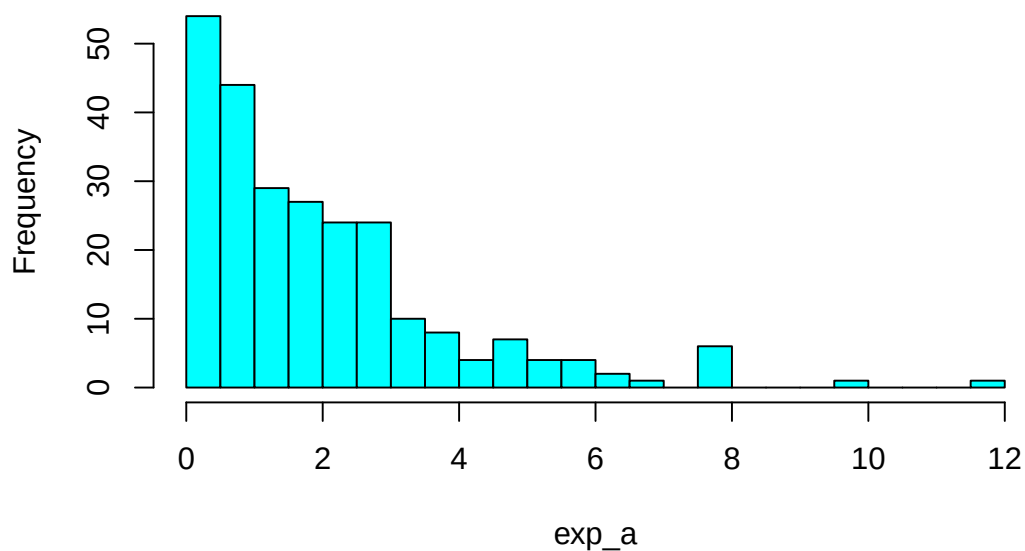
– Exponential Distribution

The exponential distribution models the time between events in a Poisson process, describing the probability of waiting a certain amount of time until an event occurs. It's often used for survival analysis and reliability studies.

$$f(x; \lambda) = \lambda e^{-\lambda x}, \quad x \geq 0$$

```
exp_a <- rexp(n = 250, rate = 0.5)
hist(exp_a, breaks = 20, col = "cyan", main = "Density Plot of Sampled Exponential Data")
```

Density Plot of Sampled Exponential Data



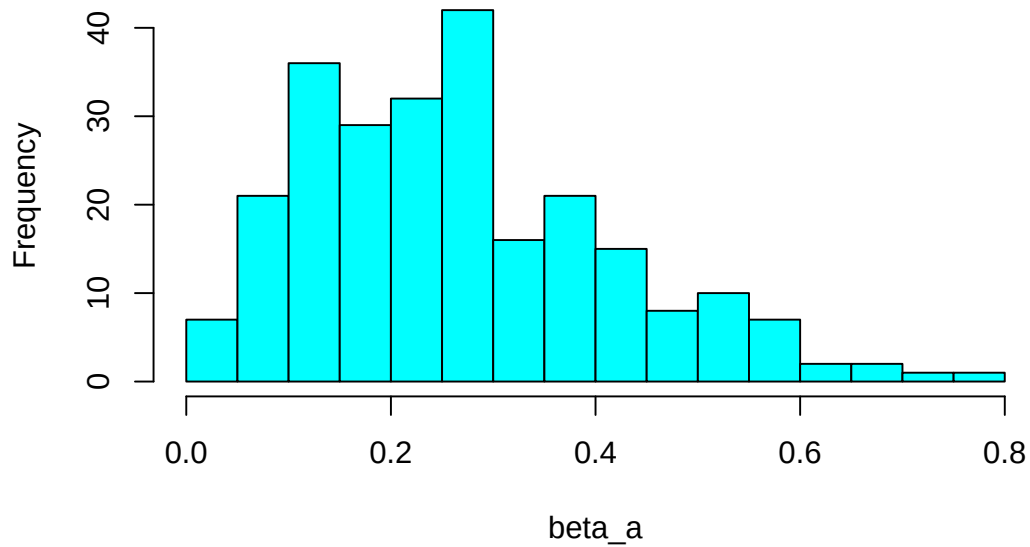
– Beta Distribution

The beta distribution is versatile and used to model data between 0 and 1, often representing proportions, probabilities, or proportions of successes in a sequence of independent Bernoulli trials.

$$f(x; \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha) \cdot \Gamma(\beta)} x^{\alpha-1} (1-x)^{\beta-1}$$

```
beta_a <- rbeta(n = 250, shape1 = 2, shape2 = 5)
hist(beta_a, breaks = 20, col = "cyan", main = "Density Plot of Sampled Beta Data")
```

Density Plot of Sampled Beta Data



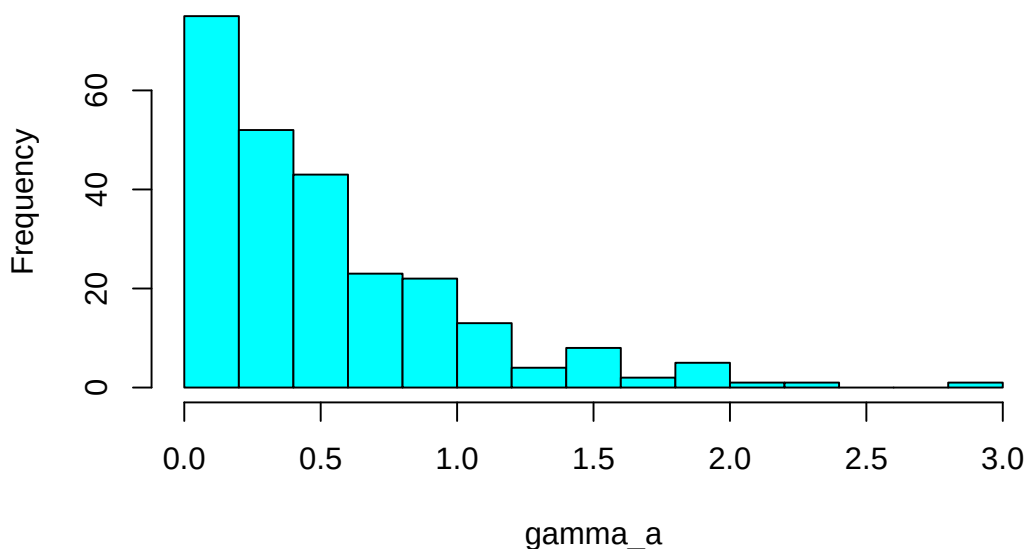
– Gamma Distribution

The gamma distribution is flexible and suitable for different variables, including modeling time to events, environmental data, and more. It is characterized by the shape (α) and scale (β) parameters and it is defined for non-negative random variables.

$$f(x; \alpha, \beta) = \frac{\beta^\alpha x^{\alpha-1} e^{-\beta x}}{\Gamma(\alpha)}$$

```
gamma_a <- rgamma(n = 250, shape = 1, rate = 2)
hist(gamma_a, breaks = 20, col = "cyan", main = "Density Plot of Sampled Gamma Data")
```

Density Plot of Sampled Gamma Data



3.2.2 The Normal Model

Recall that a random variable X has a Normal distribution if its probability density function(pdf) is of the form:

$$f(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-(x - \mu)^2 / 2\sigma^2)$$

where $\mu = E[X]$ and $\sigma^2 = Var[X]$ are the mean and variance of the pdf, and the random variable X can take values in the interval $(-\infty, \infty)$.

- For many applications we assume that a sample from an environmental continuous random variable $x_1, x_2, \dots, x_n$ has a Normal distribution if its empirical cumulative distribution function does not differ too much from the theoretical Cumulative Normal Distribution function $F(x) = P(X \leq x)$.
- Usually environmental variables are positive variables and are skewed (distribution is non-symmetric). Working with a transformed variable of the original variable X , as for example, $Y = \log(X)$ can solve the lack of normality, and Y will have a range in the interval $(-\infty, \infty)$. If Y is normally distributed, the variable X will be log-normally distributed.

3.2.2.1 Histograms, QQ-plots and PP-plots

Histograms, QQ-plots, and PP-plots are graphical tools commonly employed in statistical analysis to understand and visualize the distribution of data. A histogram provides a visual representation of the frequency or density of values within different intervals, offering insights into the shape and spread of the data. QQ-plots (Quantile-Quantile plots) serve to assess how well a data set aligns with a theoretical distribution by comparing quantiles of the observed data with those of the expected distribution. Meanwhile, PP-plots (Probability-Probability plots) focus on comparing cumulative distribution functions, aiding in the identification of deviations from expected patterns. Together, these tools offer practical means to explore, interpret, and validate the statistical properties of data sets.

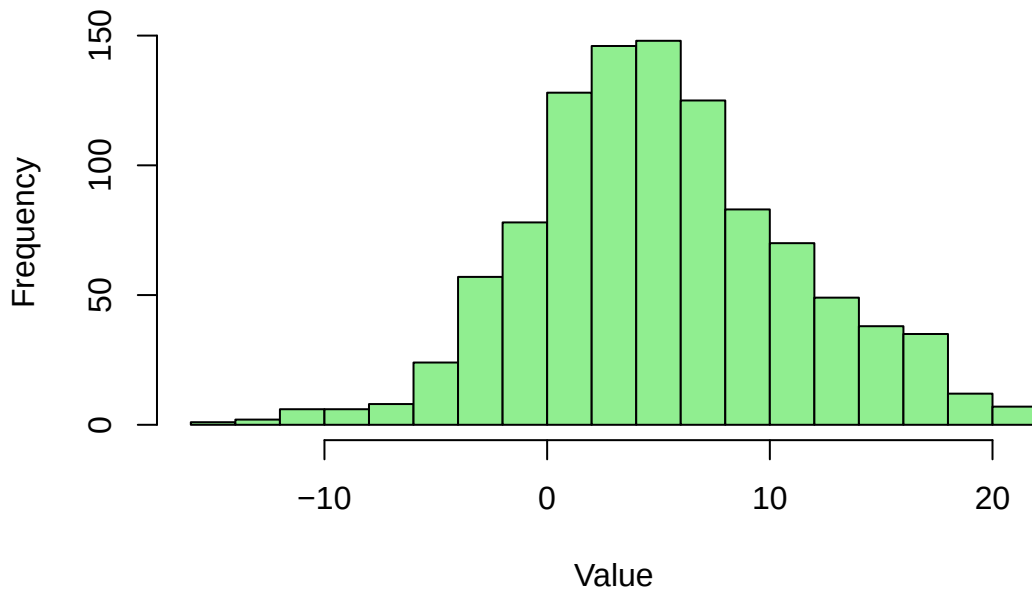
3.2.2.1.1 Histograms

Recall that a histogram is a graphical representation of the frequency distribution of the sample. Values in the y axis can be raw frequencies or relative frequencies. Below, we show the histogram for the air temperature in March for Champaign and for all months.

```
# Load weather data
data_cmi <- read.csv("data/CMI_TRH.csv")
# Do this to convert string to date format
data_cmi$date <- as.Date(data_cmi$date, format = "%m/%d/%Y")
# Subset the data for month 3
data_cmi_month3 <- subset(data_cmi, format(date, "%m") == "03")
# Create a histogram for average air temperature
hist(data_cmi_month3$avg_air_temp_c,

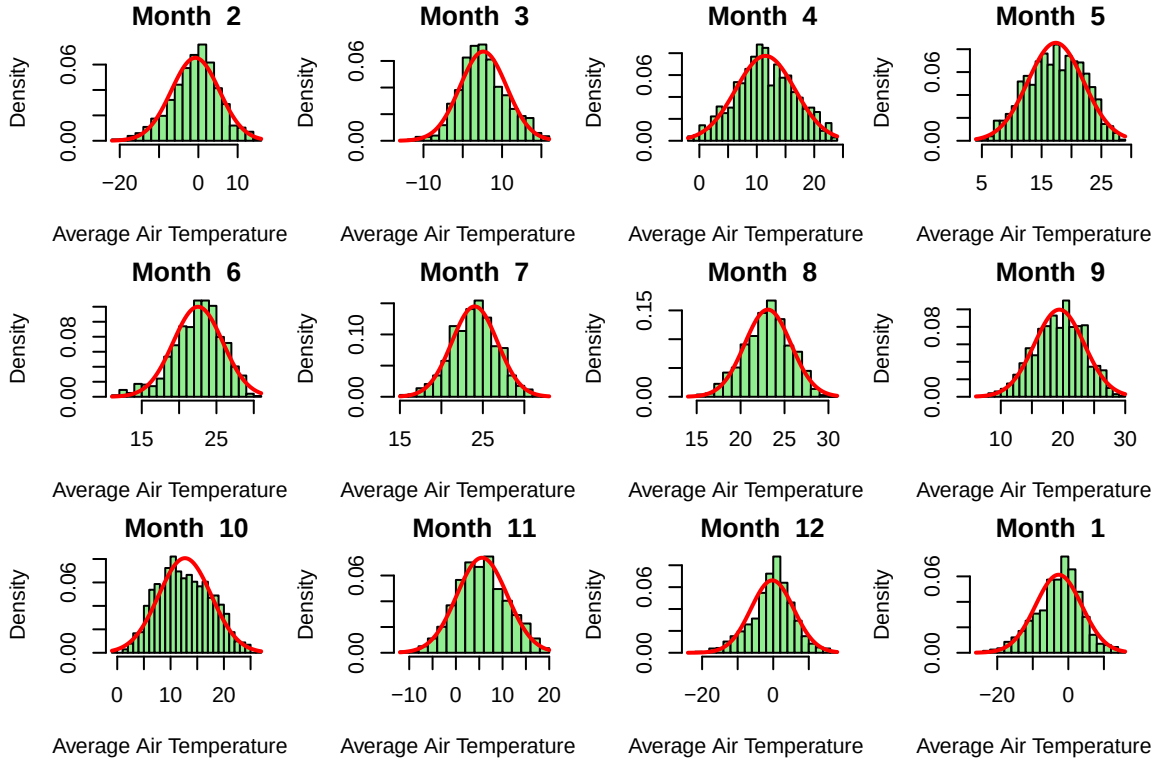
      main = "AVG Air Temperature in March",
      breaks = 20,
      xlab = "Value",
      ylab = "Frequency",
      col = "lightgreen")
```


AVG Air Temperature in March



```
# Set up a 3x4 grid for 12 plots
par(mfrow = c(3, 4), mar = c(4, 4, 2, 1))

# Loop through each month
for (m in unique(data_cmi$month)) {
  # Subset data for the current month
  month_data <- subset(data_cmi, month == m)
  # Create a histogram
  hist(month_data$avg_air_temp_c, main = paste("Month ", m), breaks=20,
        xlab = "Average Air Temperature [C]", col = "lightgreen", probability = TRUE)
  # Add a bell curve
  curve(dnorm(x, mean = mean(month_data$avg_air_temp_c), sd = sd(month_data$avg_air_temp_c))
        add = TRUE, col = "red", lwd = 2)
}
```



```
# Reset par settings
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

We can see that the average air temperatures in a month approximately follow a normal distribution. However a better visual tool for checking normality is the quantile-quantile plot (QQ-plot).

3.2.2.1.2 QQ-plots

A QQ-plot compares the quantiles of a data distribution with the quantiles of a theoretical distribution from a specified family of distributions, e.g., the standard normal distribution. In this case the y-axis is the quantiles of the data distribution, and the x-axis is the quantiles of the standard normal distribution.

Assume a sample is generated from a certain distribution. Data is sorted in increasing order $y^{(1)}, y^{(2)}, \dots, y^{(n)}$, and a plotting position is calculated for each value as:

$$\hat{P}(Y \leq y^{(i)}) = \frac{i - 0.5}{n}$$

The plotting position represents the proportion of observations below a specific data value after a continuity correction.

```
# Simulations from a normal distribution
y <- rnorm(100)
n <- length(y)
yp <- ((1:n) - 0.5)/n
#quantiles of a standard normal
zq <- qnorm(yp, mean=0, sd=1)
```

```
plot(zq, sort(y), xlab="Standard Normal Quantile", ylab="Data")
abline(mean(y), sd(y))
```

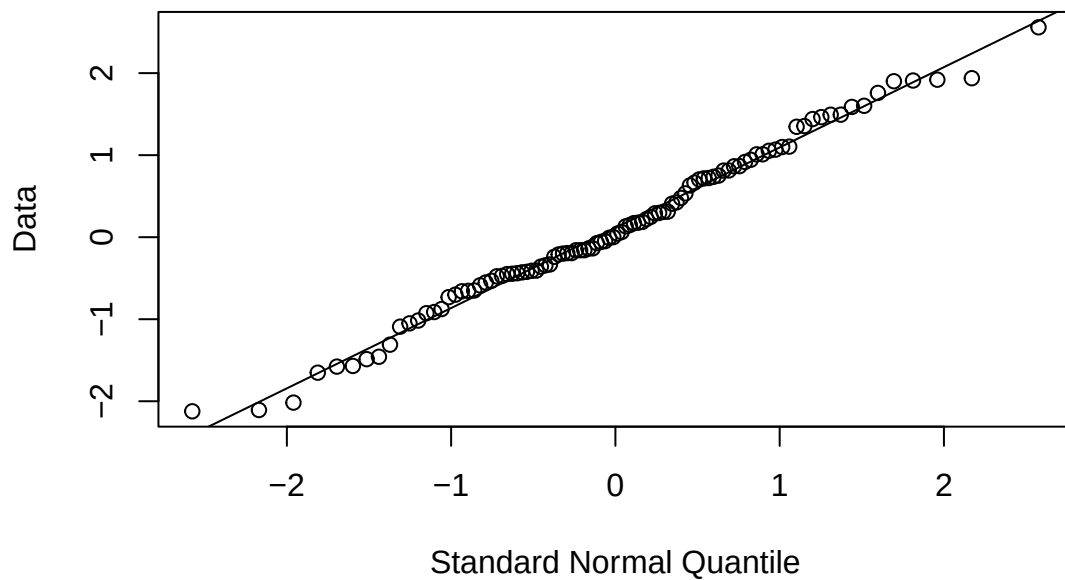


Figure 3.1: qq-plot

The above calculation is equivalent to:

```
qqnorm(y)
qqline(y)
```

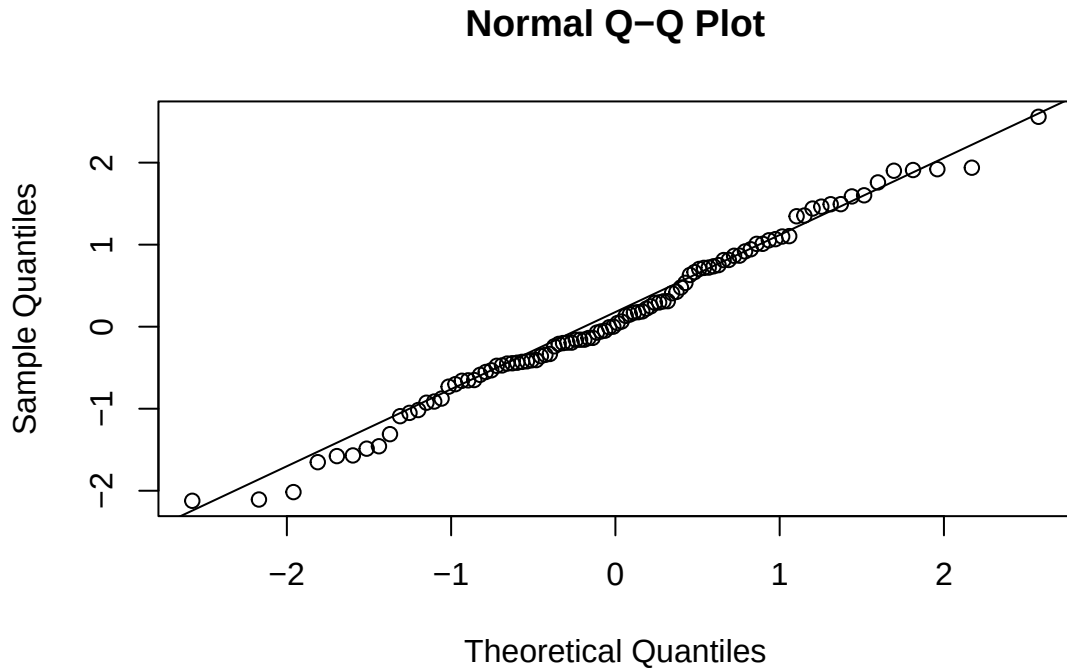


Figure 3.2: qq-plot

As seen from the QQ plot, the data points closely follow the diagonal line, which suggests that the distribution of the data approximates a normal distribution. This alignment indicates that the quantiles of our dataset match well with the quantiles of a normal distribution

3.2.2.1.3 PP-plots

A PP-plot compares the empirical cumulative distribution function of a data set with a specified theoretical cumulative distribution function $F(\cdot)$

In this case the values of theoretical distribution function $F(y^{(i)})$ are plotted against the plotting positions of the ordered data.

The construction of a QQ-plot does not require the specification of the location and the scale parameters of the distribution, while the PP-plots requires this specification to evaluate the cdf at the ordered data values.

3.2.3 The Central Limit Theorem

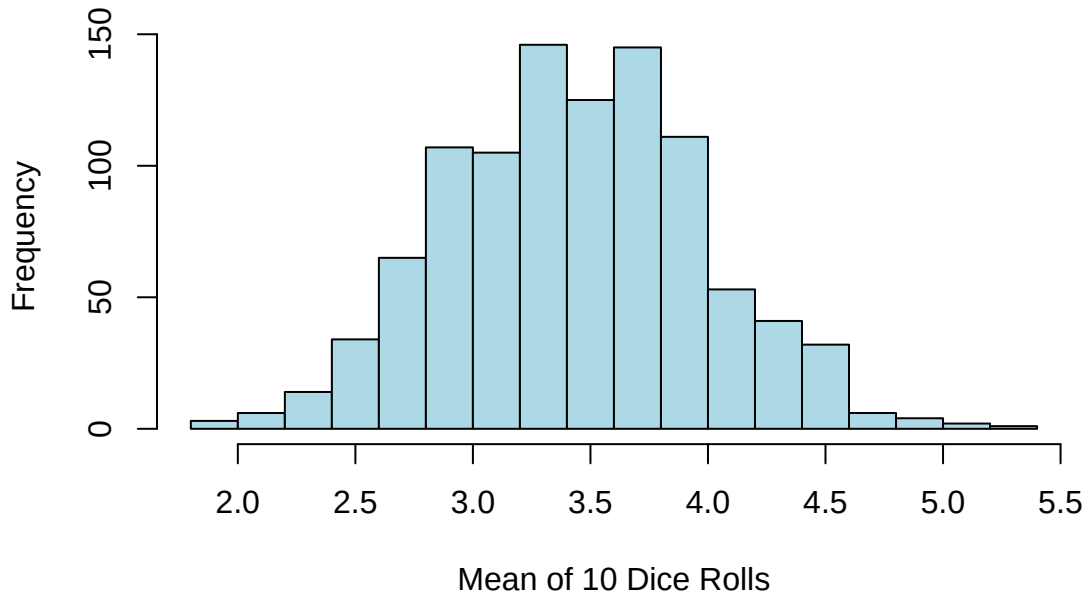
3.2.3.1 Sampling Distribution

- A statistic is a function of a sample, and since the sample values are the outcomes of a random variable, the statistic is also a random variable, for example: the sample mean ($\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$).
- Other statistics as the sample median of a sample are a function of the sample and they are considered random variables. We can talk about the sampling distribution of the sample median or any other statistic.
- According to the Central Limit Theorem, the sampling distribution of the sample mean is a Normal distribution $N(\mu, \sigma^2/n)$.
- Sampling distributions are important for statistical inference since they are used in hypothesis testing and for building confidence intervals.

The normal distribution is very important in many applications due to the Central Limit Theorem.

This theorem establishes that if you have a sample $x_1, x_2, \dots, x_n$ from a population, not necessarily normally distributed, the sum of values or sample mean (sum corrected by the sample size) is going to have a Normal distribution with mean equal to μ and variance $= \sigma^2/n$, where μ and σ^2 are the mean and variance of from the population distribution.

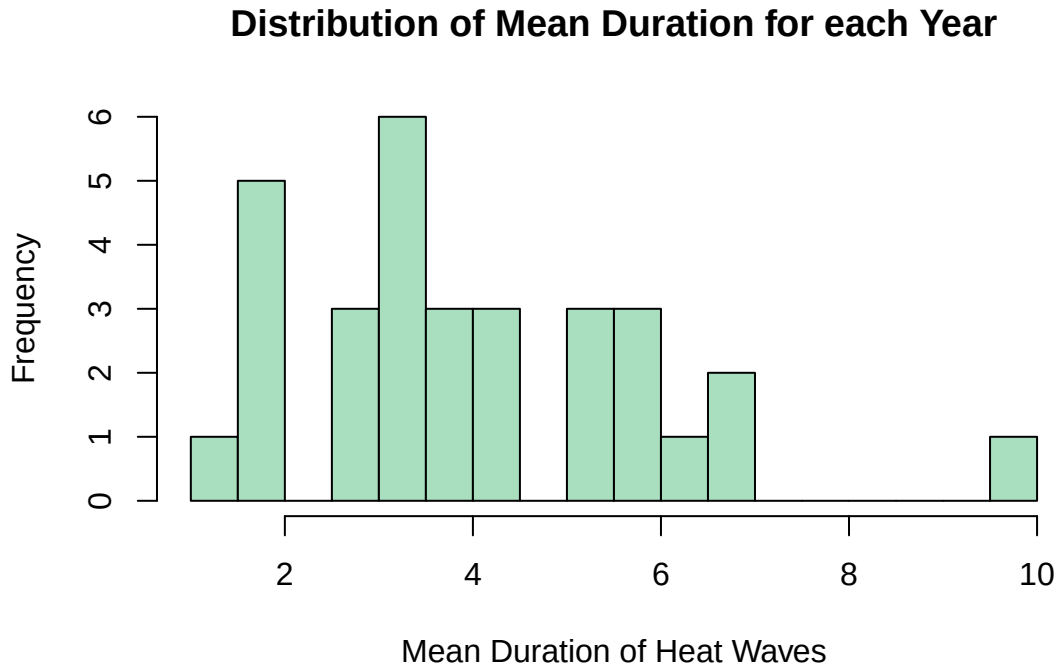
Central Limit Theorem Simulation



From figure the figure above, you can confirm the sampling distribution approximately resembles a normal distribution.

Distribution of Mean Heat Wave Duration

When plotting the histogram of the variable mean heat wave duration, the resulting figure does not show a symmetrical behavior, but rather a right skewed distribution. We can perform a hypothesis test to check whether the data sample support the normality assumption of the random variable mean heat wave duration.



3.3 Statistical Inference

Statistical inference includes all methodologies from probability theory to estimate unknown quantities of a population from a sample observed. These quantities are usually parameters from a probability model that is assumed to have generated the sample. We can use point estimation, interval estimation or hypothesis testing to get estimates the unknown parameters of a population. We will describe hypothesis testing in some detail.

3.3.1 Introduction to Hypothesis Testing

Hypothesis testing is a statistical inference approach that uses sample data to draw conclusions about either a population parameter or probability distribution. The null hypothesis (H_0), and the corresponding alternative hypothesis (H_a), representing the opposite assertion, are initially proposed. From the sample data, the procedure determines whether there is evidence to reject H_0 . If H_0 is rejected, the statistical inference favors H_a .

Motivating Example

Suppose we have a coin and we want to test if it is a fair coin. We toss it 10 times, and we only get two tails and the rest are heads. Now we can use the concept of hypothesis testing to confirm our assumption.

The null hypothesis would be that the coin is a fair coin, which means $P(H) = P(T) = 0.5$ where H represents a head and T represents a tail. Therefore, the alternative hypothesis is that the coin is not a fair coin, which is equivalent to say that $P(H) \neq P(T) \neq 0.5$. Under the null hypothesis, which is that the coin is fair, the probability of having 2 tails and 8 tails from 10 tosses is

$$P = \binom{10}{2} = 0.5^2 0.5^8 = 0.0439$$

The probability under the null hypothesis is very low. If we give a threshold, which specifies how low is “too low” for the probability that the sample occurs, we can determine if we choose to accept the null hypothesis or reject it. If our threshold is 0.05, then we will reject the null hypothesis, which means we refuse to believe that the coin is a fair coin. The reasoning is that if the coin is fair, then it would be very improbable to see samples like this.

The key concepts in hypothesis testing can be summarized as follows:

3.3.1.1 Standard errors

The term standard error is used when we refer to the standard deviation of the sampling distribution of a statistic.

**Standard error of the sample mean:

The standard error of the sample mean is the standard deviation of the sample mean denoted as: $\sigma_{\bar{x}} = \sigma / \sqrt{n}$

3.3.1.2 Type I and II errors

We normally assume that the Null Hypothesis is correct before making the test, and fix a small type I error which is the Probability of Rejecting the Null hypothesis assuming that H_0 is correct:

- Type I error (false positive): $P(\text{Rejecting } H_0 | H_0 \text{ is True}) = \alpha$
- Type II error (false negative): $P(\text{Not Rejecting } H_0 | H_0 \text{ is False}) = \beta$
- We normally control α , which is the level of significance of the test

3.3.1.3 P-value

The P-value is the probability of finding a test statistic t as extreme or more extreme than the observed value T assuming that the null hypothesis is correct. If that probability is lower than a significance level α we say that the data does not support the Null hypothesis assumption.

Example of Two-tailed One-sample Z-test

In this example we show the 2-tailed one-sample z test as an illustration of hypothesis testing. The one-sample z test is usually used to test if the sample mean is significantly different from the population mean. Z-test usually requires that the sample size is greater than 30, and the population standard deviation σ is known.

We are testing if the sample mean is significantly different from 50. So our null hypothesis is that the sample mean is not different from 50, and the alternative hypothesis is that the sample mean is different from 50.

We first generate fake data from a normal distribution with mean=55 and standard deviation of 28:

```
fake_data_z <- rnorm(n=250, mean=55, sd=28)
# print(fake_data_z)
```

We then calculate the test statistic, which in this case, is called the Z-score. The expression of the Z-score is:

$$Z = \frac{(\bar{X} - \mu)}{\sigma/\sqrt{n}}$$

```
z_score <- (mean(fake_data_z) - 50) / (28/sqrt(100))
cat('The z_score is', z_score)
```

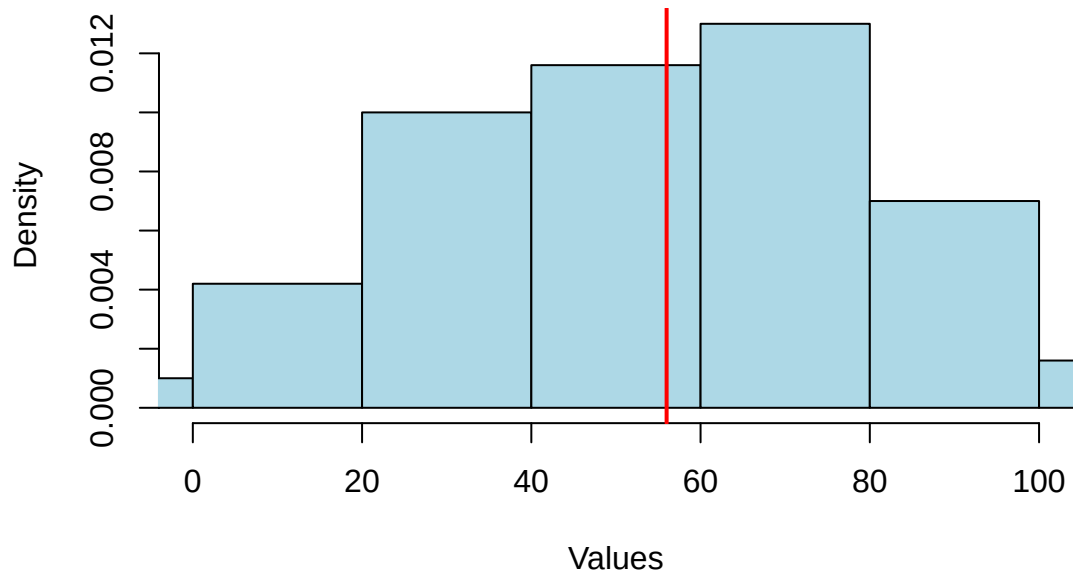
The z_score is 2.138189

We then use the decision rule to determine if we should reject the null hypothesis. The p-value is calculated as:

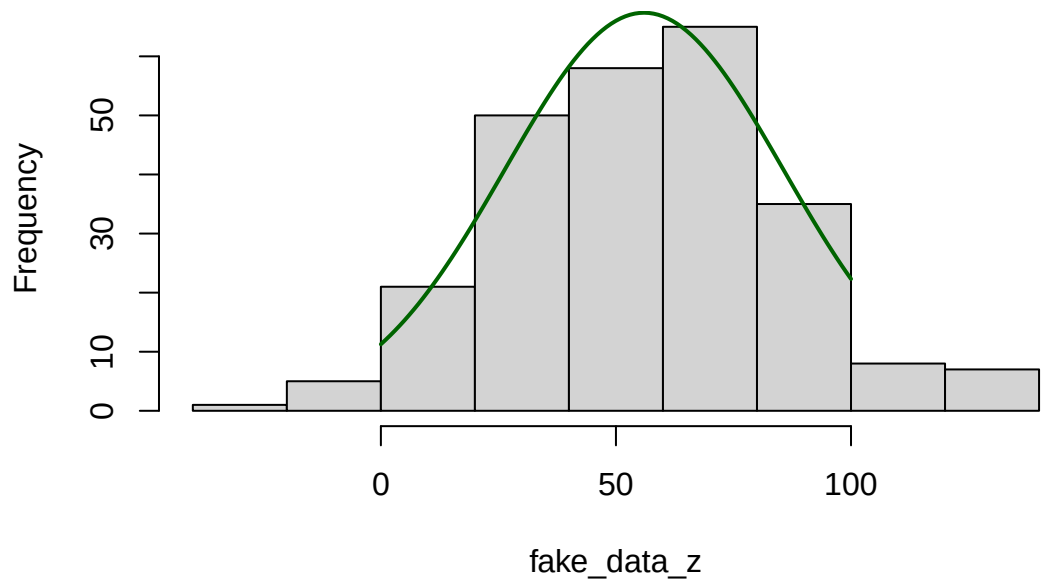
```
p_z <- 2 * (1 - pnorm(abs(z_score)))
cat('The p-value is ', p_z)
```

The p-value is 0.03250139

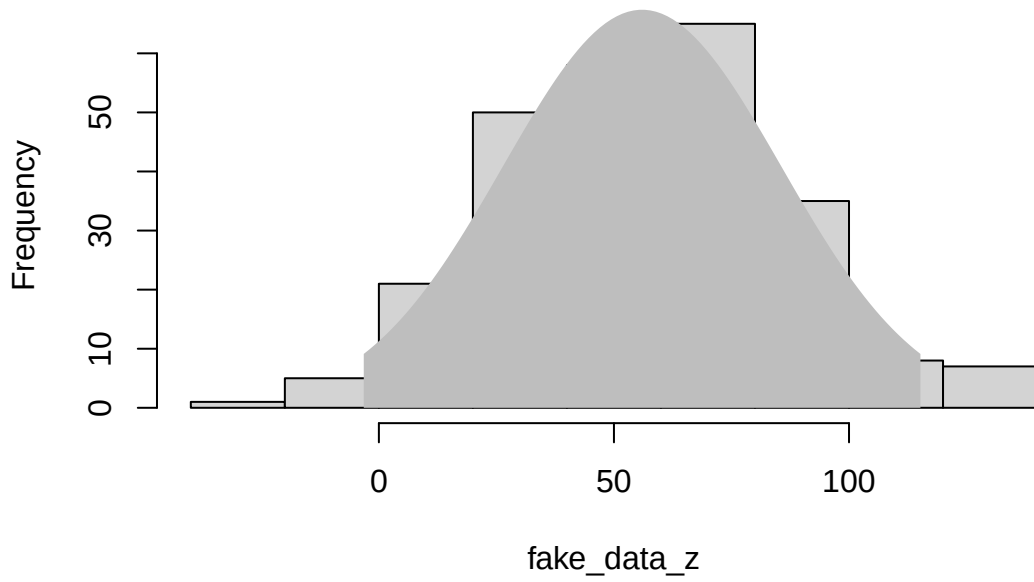
Histogram of Fake Data



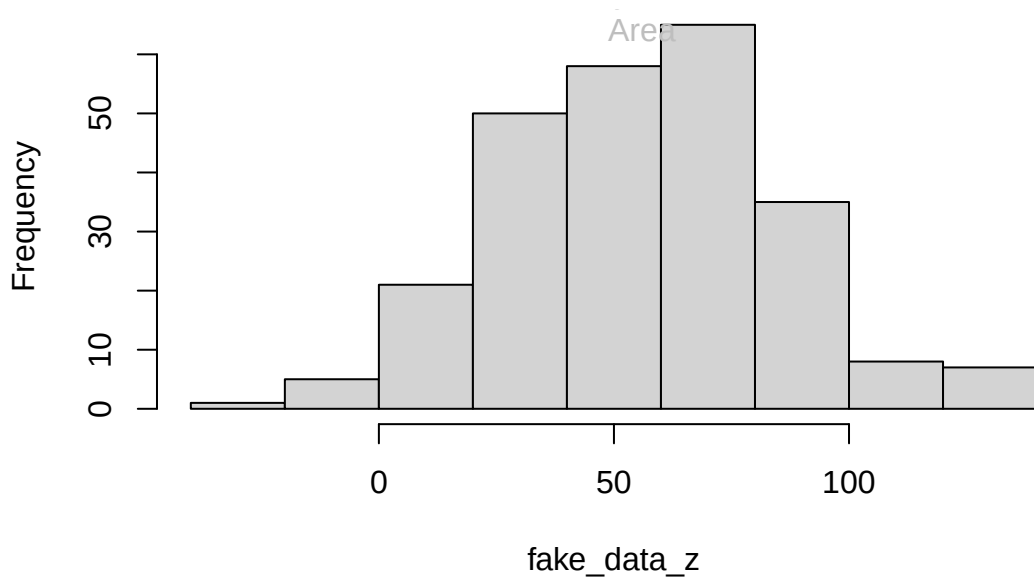
Histogram of fake_data_z

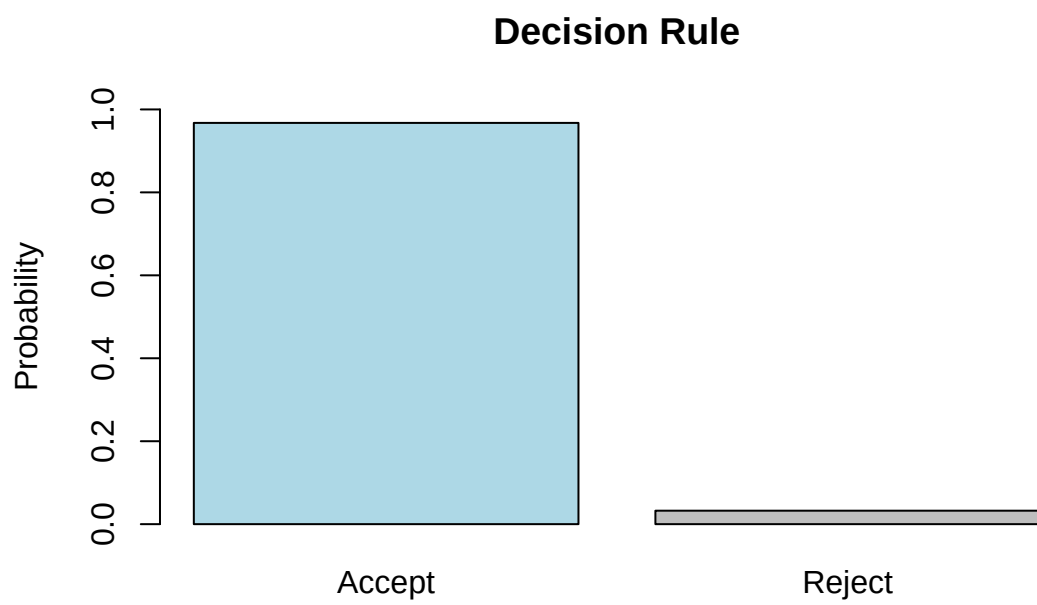


Histogram of fake_data_z



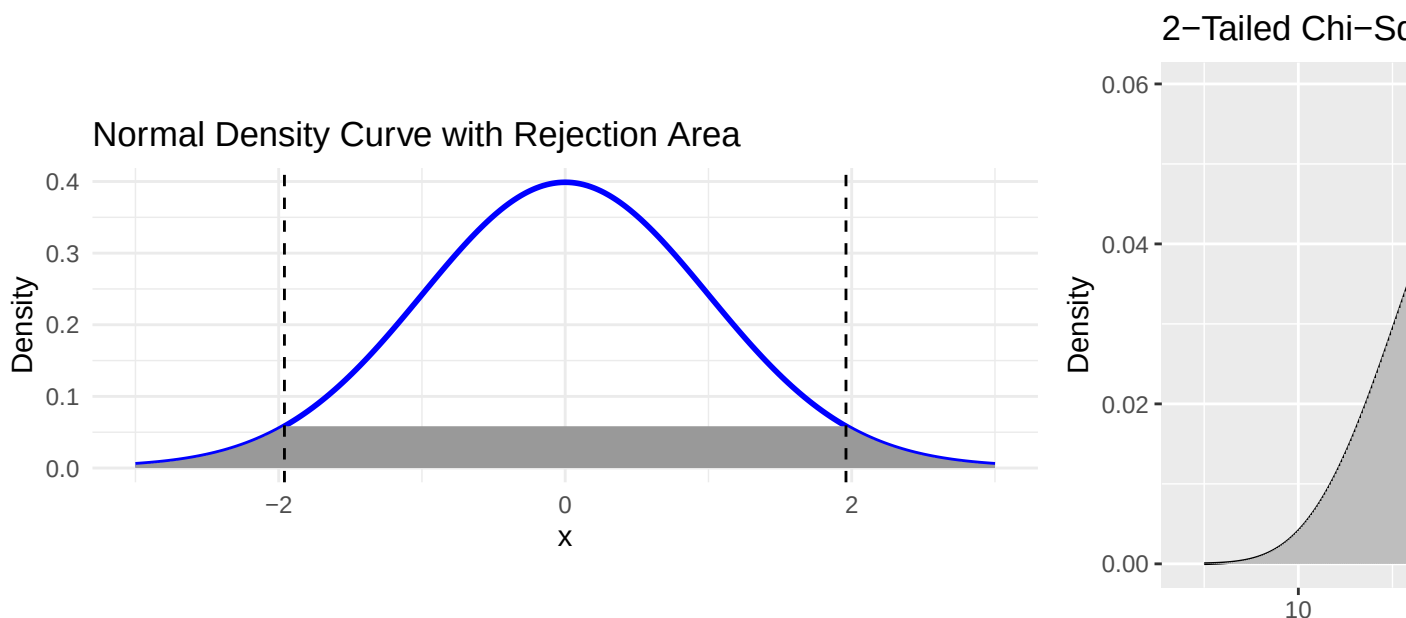
Histogram of fake_data_z





The p-value is very small and less than 0.05, so we reject the null hypothesis and claim that the sample mean is significantly different from 50. The result of the hypothesis test also matches our initial construction (we sampled from a normal distribution with mean = 55).

Warning: package 'dplyr' was built under R version 4.5.2



This is the caption for this plot

3.3.2 Testing for Normality

When using a hypothesis testing approach we use a function of the sample data (test statistic) to make a decision about an assumption on a population distribution or a population parameter.

In this case the assumption (null hypothesis H_0) assumes that the data is a sample from a Normally distributed population, while the alternative hypothesis (H_a) assumes the opposite statement.

This is:

H_0 : Data is a sample from a Normal Distribution

H_a : Data is not a sample from a Normal Distribution

3.3.2.0.1 Goodness for Fit tests for the Normal Distribution

- Shapiro-Wilks: The statistic W_n is the ratio of the best estimator of the variance (based on the square of a linear combination of the order statistics) to the corrected sum of squares estimator of the variance. Values of W_n are between 0 and 1.

$$W_n = \frac{(\sum_{i=1}^n a_i x_{(i)})^2}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Small values of the statistics lead to a rejection of the null hypothesis.

3.3.2.0.2 Tests based on the Empirical Distribution Function (EDF Tests):

These tests measure the discrepancy between $F(t)$ and $\hat{F}(t)$.

- Kolmogorov-Smirnoff (D)
- Anderson Darling (Q)
- Cram'er von-Mises (W^2)

All these statistics measure the departures of the ECDF from the theoretical CDF. Large values of these statistics would lead to the rejection of the Null hypothesis.

For all tests a p-value is calculated. Suppose that under the null hypothesis we assume that the collected sample comes from population with Normal distribution. If the p value is less than the chosen significance level (α), then the null hypothesis is rejected and there is evidence that the data tested are not coming from normally distributed population. On the other hand, if the p value is greater than the chosen significance level, then the null hypothesis (that the data came from a normally distributed population) can not be rejected.

3.3.3 Location Tests

3.3.3.0.1 One-Population location tests

In all cases an independent and identically distributed (iid) sample of a population is available and we want to test a particular assumption about the location parameter μ_0

Test	Population Assumption	Location parameter μ_0
Student's t	<i>iid</i> , $N(\mu, \sigma^2)$	mean (same as median)
Signed Rank	<i>iid</i> , symmetric distribution	median
Sign	<i>iid</i>	median

3.3.3.0.2 Hypothesis Testing on a Population Mean μ

We assume that we have independent samples from a normal population with mean μ and variance σ^2 . We want to test the hypothesis $H_0 : \mu = \mu_0$.

The alternative hypothesis can be:

- $H_a : \mu \neq \mu_0$ (Two-tailed test)
- $H_a : \mu > \mu_0$ (Right-tailed test)
- $H_a : \mu < \mu_0$ (Left-tailed test)

If the standard deviation σ is unknown we use the t-statistic:

$$T = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

where $\bar{x}$ is the sample mean and s the sample standard deviation.

Under the normality and independence assumptions $T \sim$ t-student distribution with $n - 1$ degrees of freedom.

Given a significance level α we reject the null hypothesis if:

- P-value: $P(|t| \geq |T| | H_0) < \alpha$ (two-tail test)
- P-value: $P(t \geq T | H_0) < \alpha$ (right-tail test)
- P-Value: $P(t \leq T | H_0) < \alpha$ (left-tail test)

where $t \sim$ t-student($n - 1$)

Example of One-sample t-test in R

Suppose we want to test if a list of weights normally distributed has a mean of 150 pounds

```
# Generate samples for testing
weights_1 <- round(rnorm(10, mean = 150, sd = 20))

# print out the samples
print(weights_1)
```

```
[1] 161 175 141 148 151 145 141 131 145 123
```

```
# Perform one-sample t-test
t_test_1 <- t.test(weights_1, mu = 150)

# Print the results of the t-test
print(t_test_1)
```

One Sample t-test

```
data: weights_1
t = -0.84861, df = 9, p-value = 0.4181
alternative hypothesis: true mean is not equal to 150
95 percent confidence interval:
 135.7036 156.4964
sample estimates:
mean of x
 146.1
```

You can observe that the test gives a p-value of 0.33, which allows us to accept the null hypothesis and claim that the mean is 150.

However, if we generate the samples with mean=120, the t test will give us a p-value that rejects the null hypothesis

```
# Generate samples for testing
weights_2 <- round(rnorm(10, mean = 120, sd = 20))
# print out the samples
print(weights_2)
```

```
[1] 91 109 94 166 127 138 110 90 110 144
```

```
# Perform one-sample t-test
t_test_2 <- t.test(weights_2, mu = 150)
# Print the results of the t-test
print(t_test_2)
```

One Sample t-test

```
data: weights_2
t = -4.0129, df = 9, p-value = 0.003051
alternative hypothesis: true mean is not equal to 150
95 percent confidence interval:
 99.80447 135.99553
sample estimates:
mean of x
 117.9
```


3.3.3.0.3 Two-population location tests

The main interest is on comparing the location parameters from two population distributions μ_1 and μ_2 .

A common hypothesis is $H_0 : \mu_1 = \mu_2$.

The alternative hypothesis can be:

- $H_a : \mu_1 \neq \mu_2$ (Two-tail tests)
- $H_a : \mu_1 > \mu_2$ (Right-tail test)
- $H_a : \mu_1 < \mu_2$ (Left-tail test)

Assuming that the we have two iid samples from populations 1 and 2 with unknown variances σ_1^2 and σ_2^2 ; and sample means $\bar{x}_1$ and $\bar{x}_2$ and sample variances s_1^2 and s_2^2 , the following test statistics can be used:

Variances	Population Assumption	test-statistic under $H_0 : \mu_1 = \mu_2$	statistic's Distribution
Equal	$N(\mu_1, \sigma^2)$ and $N(\mu_2, \sigma^2)$	$T = \frac{\bar{x}_1 - \bar{x}_2 - (\mu_1 - \mu_2)}{s\sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}$	t-Student with $n_1 + n_2 - 2$ df
Different	$N(\mu_1, \sigma_1^2)$ and $N(\mu_2, \sigma_2^2)$	$T = \frac{\bar{x}_1 - \bar{x}_2 - (\mu_1 - \mu_2)}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$	t-Student (Satterthwaite approx for df)
Equal	Distributions have equal shape, different location	Wilcoxon's Rank-Sum test	Exact for small n , Normal approx. for large n

For a given level of significance α since the t-Student is a symmetric distribution one can say that:

$$P(|T| \leq t_{df;\alpha/2}) = 1 - \alpha$$

By inverting this probability equation we get an expression for the $(1 - \alpha)$ 100% Confidence Interval:

$$\bar{x}_1 - \bar{x}_2 - s\sqrt{\frac{1}{n_1} + \frac{1}{n_2}} \times t_{df;\alpha/2} \leq \mu_1 - \mu_2 \leq \bar{x}_1 - \bar{x}_2 + s\sqrt{\frac{1}{n_1} + \frac{1}{n_2}} \times t_{df;\alpha/2}$$

s is a pooled sample variance estimate assuming σ_1^2 and σ_2^2 are equal:

$$s = \sqrt{\frac{s_1^2(n_1 - 1) + s_2^2(n_2 - 1)}{n_1 + n_2 - 2}}$$

Example of a 2-sample t-test

```
# Perform a two-sample t-test
t.test(weights_1, weights_2, var.equal = FALSE)
```

Welch Two Sample t-test

```
data: weights_1 and weights_2
t = 3.0568, df = 14.358, p-value = 0.00833
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 8.45953 47.94047
sample estimates:
mean of x mean of y
 146.1      117.9
```

You can see that the 2-sample t-test shows a p-value of 0.015, which means we should reject the null hypothesis and claim the mean weights for the 2 groups are different.

3.3.3.1 Confidence Interval

The previous equation can lead to a wrong interpretation of the confidence interval, since it might appear like the probability of the parameter $\mu_1 - \mu_2$ lying between the lower and upper limit indicated above is 95%. **That interpretation is incorrect, because according to the frequentist framework, the parameter $\mu_1 - \mu_2$ is not a random variable since it is a population parameter.**

The limits of the Confidence Interval are random numbers because they depend on the sample and you will have different values if you have a different samples. The correct interpretation uses the idea of repeated sampling: If we have 100 hundred independent samples from a population and calculate a CI for each sample, 95 of the 100 CIs would contain the true population statistic $\mu_1 - \mu_2$.

3.3.4 Measures of Linear Association

In climate data analysis, we frequently need to check whether two continuous variables X and Y are independent. For example, we want to test the hypothesis of whether mean daily ozone concentration during a certain time interval for a given location is independent of the average wind speed measured at the same location or a close by location.

3.3.4.1 Pearson's Correlation Significance Test

The Pearson correlation coefficient ρ is a simple and straight-forward indicator of the linear association between two variables. The magnitude of the correlation is proportional to the strength of linear dependence. This dependence can be positive or negative. Recall the expression for the sample estimate of the Pearson correlation coefficient between X and Y is:

$$\hat{\rho}^2 = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}}$$

Assuming that the two variables X and Y have a Normal distribution, we can test the hypothesis of no significant linear association between X and Y , that is $H_o : \rho = 0$. Assuming that the null hypothesis is correct, the following test statistic follows a t-distribution with $n - 2$ degrees of freedom:

$$T = \frac{\hat{\rho}\sqrt{n-2}}{\sqrt{1-\hat{\rho}^2}}$$

We would normally perform a 2-sided t-test, which means we should reject the null hypothesis of linear association if

$$|T| \leq t_{df;\alpha/2}$$

If two variables are independent the Pearson correlation coefficient ρ is negligible. However, if the correlation coefficient is zero or close to zero, we can not say that the variables are independent, since they might have a different kind of association other than linear. In other words, if two variables are independent, their correlation coefficient is equal to zero; but if the correlation is zero, this does not imply that the two variables are independent.

3.3.4.2 Chi-sq Test for Independence

The chi-square test for independence is a statistical method used to determine if there is a significant association between two categorical variables. It assesses whether the observed distribution of frequencies in a contingency table differs from what would be expected if the variables were independent. In other words, it helps us understand if changes in one variable are related to changes in another. The test is based on the chi-square statistic, which compares the observed and expected frequencies. If the calculated chi-square value is sufficiently large, it suggests a significant association between the variables. This test is widely employed in fields such as sociology, biology, and market research to explore relationships between categorical variables and make informed inferences about populations based on sample data.

	Truck	Sedan
Male	24	51
Female	13	42

```
# Perform the chi-square test for independence
result <- chisq.test(fake_data)

# Print the test results
cat("Chi-Square Test for Independence Results:\n")
```

Chi-Square Test for Independence Results:

```
print(result)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: fake_data
X-squared = 0.71804, df = 1, p-value = 0.3968
```

We can see that because of the high p-value we don't reject the null hypothesis and we should claim that there is no significant association between gender and types of cars they drive.

4 Non-Parametric Methods for Statistical Inference

In the previous chapter we discussed several statistical tests applied to a sample from a population. The sample is assumed to be independent and identically distributed, and all values are assumed to be a random sample from a Normal distribution.

Non-parametric methods in statistics provide a versatile and robust alternative to traditional parametric techniques, particularly when the assumptions of normality or other underlying distributions cannot be met. These methods are characterized by their flexibility and ability to analyze data without making specific assumptions about the population distribution. Examples of non-parametric methods include the Wilcoxon signed-rank test, which assesses differences between paired observations, the Mann-Whitney U test, used for comparing two independent groups, and the Kruskal-Wallis test, which extends the comparison to multiple groups. These techniques are valuable tools for researchers and analysts working with real-world data, as they offer reliable solutions in scenarios where parametric methods may fall short.

4.1 Introduction to One-Sample Rank Tests

One common scenario where rank tests shine is when dealing with skewed or non-normally distributed data. In such cases, traditional parametric tests like the t-test or ANOVA may produce unreliable results. Rank tests offer a solution by focusing on the ranks or orders of observations, making them robust to distributional assumptions.

For example, consider a study evaluating the effectiveness of a new medication. The response variable is the level of pain relief reported by patients on a scale from 1 (no relief) to 10 (complete relief). The data collected may not follow a normal distribution, and the scale itself is ordinal rather than continuous. In this situation, a rank-based test like the Wilcoxon signed-rank test for paired data or the Mann-Whitney U test for independent samples can provide a more appropriate analysis, comparing the central tendency of pain relief between groups or before and after treatment without relying on distributional assumptions.

In essence, rank tests offer a valuable tool for statisticians and researchers when dealing with real-world data that doesn't conform to the idealized assumptions of parametric methods.

They provide a robust and reliable means of drawing valid conclusions, even in challenging situations.

4.1.1 Wilcoxon Signed-Rank Test

The signed rank tests uses the difference between the sample values from the proposed median $x_i - \mu_0$, but also uses the size of the difference to calculate a rank. This tests assumes that the sample is coming from a symmetrical distribution.

Example

If the differences of the sample from the proposed median are: -1.6, 2.3, -5.1, 3.9, the sorted values without considering the signs are: -1.6, 2.3, 3.9, -5.1, with ranks 1, 2, 3 and 4. The values of the signed rank test statistic (TS) are the ranks for the **negative values**, this is: $TS = 1 + 4 = 5$. Other options are to use the positive values: $TS = 2 + 3$.

- A large value of TS would indicate more observations **below the median** than expected under H_0 .
- When ties are present in the data, they are removed and the sample size is adjusted accordingly.
- When n is moderately large, the TS statistic has an Normal distribution under the Null hypothesis, with mean and variance:

$$E[TS] = \frac{n(n+1)}{4}$$

$$Var[TS] = \frac{n(n+1)(2n+1)}{24}$$

Assume that $TS = t$. The Signed Rank Test rejects the Null hypothesis for a two-tailed test if:

- $P(TS \leq t) \leq \alpha/2$
- $P(TS \geq t) \leq \alpha/2$

In R, the function `wilcox.test(.)` can be used for testing the location parameter for a single population, using the parameter $\mu = \dots$ in the function call.

Important: *The sign test does not need any assumption about the shape of the distribution of the data.* The Wilcoxon Signed Rank test requires that the data has a symmetric distribution.

Example

An example goes here

```
library(dplyr)
```

Warning: package 'dplyr' was built under R version 4.5.2

```
library(tidyverse)
```

Warning: package 'ggplot2' was built under R version 4.5.2

Warning: package 'tibble' was built under R version 4.5.2

Warning: package 'tidyr' was built under R version 4.5.2

Warning: package 'readr' was built under R version 4.5.2

Warning: package 'purrr' was built under R version 4.5.2

Warning: package 'stringr' was built under R version 4.5.2

Warning: package 'forcats' was built under R version 4.5.2

Warning: package 'lubridate' was built under R version 4.5.2

```
library(ggplot2)  
library(descstatsr)
```

Warning: package 'descstatsr' was built under R version 4.5.3

```
library(goftest)
```

Warning: package 'goftest' was built under R version 4.5.2

```
library(BSDA)
```

Warning: package 'BSDA' was built under R version 4.5.3

```
hwf<-read.csv("data/hw_factors.csv")
# Mean Intensity
hwf$period<- ifelse(hwf$year<2005,"Before 2005","After 2005")
hwf %>% filter(year<2005) %>%
  summarize(mean_int_b2005=mean(mean_intensity))
```

```
mean_int_b2005
1      4.356973
```

```
hwf %>% filter(year>=2005) %>%
  summarize(mean_int_a2005=mean(mean_intensity))
```

```
mean_int_a2005
1      3.771778
```

```
# Mean duration
hwf %>% filter(year<2005) %>%
  summarize(mean_dur_b2005=mean(mean_duration))
```

```
mean_dur_b2005
1      3.373333
```

```
hwf %>% filter(year>=2005) %>%
  summarize(mean_dur_a2005=mean(mean_duration))
```

```
mean_dur_a2005
1      3.462054
```

```
hwf1<-hwf %>% filter(year<2005)
hwf2<-hwf %>% filter (year >=2005)

# One-sample t-test for mean intensity
t.test(hwf1$mean_intensity,mu=5,alternative="l")
```

One Sample t-test

data: hwf1\$mean_intensity


```

t = -1.1072, df = 14, p-value = 0.1434
alternative hypothesis: true mean is less than 5
95 percent confidence interval:
    -Inf 5.379885
sample estimates:
mean of x
    4.356973

```

```
t.test(hwf2$mean_intensity,mu=5, alternative="l")
```

One Sample t-test

```

data: hwf2$mean_intensity
t = -3.2016, df = 15, p-value = 0.002972
alternative hypothesis: true mean is less than 5
95 percent confidence interval:
    -Inf 4.444288
sample estimates:
mean of x
    3.771778

```

```

# Two-sample t-test for mean intensity
# By default variances are not assumed equal
t.test(hwf1$mean_intensity,hwf2$mean_intensity)

```

Welch Two Sample t-test

```

data: hwf1$mean_intensity and hwf2$mean_intensity
t = 0.84076, df = 24.525, p-value = 0.4086
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
    -0.8497143  2.0201043
sample estimates:
mean of x mean of y
    4.356973  3.771778

```

```

# Sign-test (One-sample) for mean duration
SIGN.test(hwf1$mean_duration, md = 3.5, alternative='two.sided')

```

One-sample Sign-Test

```
data: hwf1$mean_duration
s = 5, p-value = 0.3018
alternative hypothesis: true median is not equal to 3.5
95 percent confidence interval:
 2.148474 3.940611
sample estimates:
median of x
      3.2
```

Achieved and Interpolated Confidence Intervals:

	Conf.Level	L.E.pt	U.E.pt
Lower Achieved CI	0.8815	2.8333	3.6667
Interpolated CI	0.9500	2.1485	3.9406
Upper Achieved CI	0.9648	2.0000	4.0000

```
SIGN.test(hwf2$mean_duration, md = 3.5, alternative='two.sided')
```

One-sample Sign-Test

```
data: hwf2$mean_duration
s = 7, p-value = 1
alternative hypothesis: true median is not equal to 3.5
95 percent confidence interval:
 2.536947 4.120687
sample estimates:
median of x
      3.416667
```

Achieved and Interpolated Confidence Intervals:

	Conf.Level	L.E.pt	U.E.pt
Lower Achieved CI	0.9232	2.5714	4.0000
Interpolated CI	0.9500	2.5369	4.1207
Upper Achieved CI	0.9787	2.5000	4.2500

```
# Wilcoxon Signed-Rank test (One-sample) for mean duration

#Before 2005
wilcox.test(hwf1$mean_duration,mu=3.5)
```

```
Warning in wilcox.test.default(hwf1$mean_duration, mu = 3.5): cannot compute
exact p-value with ties
```

Wilcoxon signed rank test with continuity correction

```
data: hwf1$mean_duration
V = 44, p-value = 0.3764
alternative hypothesis: true location is not equal to 3.5
```

```
#After 2005
wilcox.test(hwf2$mean_duration,mu=3.5)
```

```
Warning in wilcox.test.default(hwf2$mean_duration, mu = 3.5): cannot compute
exact p-value with ties
```

```
Warning in wilcox.test.default(hwf2$mean_duration, mu = 3.5): cannot compute
exact p-value with zeroes
```

Wilcoxon signed rank test with continuity correction

```
data: hwf2$mean_duration
V = 54.5, p-value = 0.7753
alternative hypothesis: true location is not equal to 3.5
```

4.2 Rank Tests with Two or More Samples

Assume we have data on precipitation projections from two Global Circulation Models (GCM1 and GCM2) for the period 2070- 2100 (last 30 years of this century), and we want to test the hypothesis that the location parameters for the two distributions ($\tilde{\mu}_1$ and $\tilde{\mu}_2$) are the same. We assume that the two samples from models GCM1 and GCM2 are not independent samples, since the models essentially similar differential equations to represent the climate-land-ocean interactions.

We can calculate the differences between each pair of observations for every time step $x_i - y_i$, and apply the Wilcoxon Rank test or the Sign test.

For a comparison of the location parameters among 3 or more populations we can use the Kruskal-Wallis test. The parametric parallel of this test is the one-way ANOVA model that will be discussed later.

4.2.1 Wilcoxon Rank-Sum test

We want to show that observations from one population $x_1, x_2, \dots, x_{n_1}$ tend to be larger (or smaller) than observations from another population $y_1, y_2, \dots, y_{n_2}$.

It is assumed that the distribution F_1 of X and F_2 of Y are identical in shape and only differ by location. This is equivalent to say that there is a shift in location.

In this case we set up the hypothesis (more general set up):

$$H_0 : F_1 = F_2 \text{ vs. } H_a : F_1 > F_2$$

(one-sided) or

$$H_0 : F_1 = F_2 \text{ vs. } H_a : F_1 > F_2 \text{ or } H_a : F_1 < F_2$$

(two-sided).

If the data support the alternative hypothesis $H_A : F_1 < F_2$ we can say that random variable Y has **stochastic dominance** over random variable X .

The steps for the test are as follows (assume no ties are present): 1. Rank all $N = n_1 + n_2$ observations in ascending order 2. Sum the ranks of the x 's ($W_1 = \omega_1$) and the ranks of the y 's ($W_2 = \omega_2$) 3. Reject H_0 if ω_1 is large and equivalently ω_2 is small.

The distributions of W_1 and W_2 are not the same when the sample sizes n_1 and n_2 are not equal.

The Wilcoxon Rank Sum Test, also known as the Mann-Whitney U test, is a non-parametric test used to assess whether there is a difference between two independent groups. It doesn't assume normality and is appropriate when the data may not meet the assumptions of a t-test.

Example An example goes here

4.2.2 Wilcoxon rank sum test and Mann-Whitney test

The **Mann-Whitney U test** is another non-parametric test available when you want to compare two distributions. The probability distribution of the random variable W (sum of the ranks) is tabulated. The *Mann-Whitney test* statistic U is related to the Wilcoxon rank sum test. In the Mann-Whitney test, the steps are as follows:

1. Calculate u_1 = number of pairs in which $x_i > y_j$ for all possible pairs i and j ; and u_2 = number of pairs in which $x_i < y_j$ for all possible pairs i and j . There are $n_1 n_2$ possible pairs and $u_1 + u_2 = n_1 n_2$.
2. Reject H_0 if u_1 is large or u_2 is small.

The Wilcoxon rank sum test and Mann-Whitney test are related in the following way:

$$u_1 = \omega_1 - \frac{n_1(n_1 + 1)}{2} \text{ and } u_2 = \omega_2 - \frac{n_2(n_2 + 1)}{2}$$

The upper tail probability distributions for the U and W statistics under H_0 are tabulated. An α -level test rejects H_0 if $P\text{-value} \leq \alpha$

Example Compare mean heat wave's duration before and after 2005

We want to check whether there has been a shift in the distributions of the mean heat wave's duration after 2005.

We will use the function `wilcox.test(x,y)`, where x is the sample vector from the random variable X , and y is the sample vector from random variable Y .

```
wilcox.test(hwf1$mean_duration,hwf2$mean_duration)
```

```
Warning in wilcox.test.default(hwf1$mean_duration, hwf2$mean_duration): cannot  
compute exact p-value with ties
```

Wilcoxon rank sum test with continuity correction

```
data: hwf1$mean_duration and hwf2$mean_duration  
W = 110, p-value = 0.7052  
alternative hypothesis: true location shift is not equal to 0
```

For a comparison of the location parameters among 3 or more populations we can use the Kruskal-Wallis test. The parametric parallel of this test is the one-way ANOVA model that will be discussed later.

4.2.3 Kruskal-Wallis test

The Kruskal-Wallis test is a powerful non-parametric statistical test designed to address situations where the assumptions of normality and homogeneity of variances required by traditional analysis of variance (ANOVA) cannot be met. This test is particularly useful when you have three or more independent groups or conditions and you want to determine whether there are statistically significant differences in the central tendencies of these groups.

In cases where the data is not normally distributed or the group variances are unequal, the Kruskal-Wallis test provides a reliable alternative to traditional ANOVA. Instead of relying on means and variances, the Kruskal-Wallis test operates on the ranks of the data, making it robust to deviations from normality and homoscedasticity.

Imagine a clinical trial comparing the pain relief efficacy of three different treatments. Patients' pain relief scores may not follow a normal distribution, and the variability in responses between treatments could be quite different. In this scenario, the Kruskal-Wallis test allows researchers to determine whether there are statistically significant differences in pain relief among the treatments without the need for stringent parametric assumptions.

In summary, the Kruskal-Wallis test is a valuable tool in situations where you need to compare multiple groups and the underlying data does not meet the assumptions of ANOVA. It provides a reliable means of detecting differences in central tendencies across groups while accommodating the realities of real-world data.

Example Here it goes an example of the Kruskal-Wallis test

4.3 Non-parametric correlation measures

4.3.1 The Spearman's rank correlation

The Spearman's correlation measures the strength of a monotonic relationship between two variables. This is a non-parametric measure of correlation and it is calculated as the Pearson correlation coefficient between the ranks of the two variables:

$$r_{s_{xy}} = \frac{\text{cov}(\text{Rank}(x), \text{Rank}(y))}{S_{\text{Rank}(x)} \times S_{\text{Rank}(y)}}$$

4.4 Kendal's rank correlation

It is more robust and efficient than the Spearman's correlation, but more difficult to interpret: Considering all possible pairs (i, j) from the sample. The Kendall's τ quantifies the difference between the percentage of all discordant and concordant pairs in the data:

$$\tau_{xy} = \frac{2}{n(n-1)} \sum_{i < j} \text{sign}(x_i - x_j) \times \text{sign}(y_i - y_j)$$

4.5 Bootstrapp Confidence Intervals

This section discussed rank-based nonparametric methods. Another class of non-parametric methods, known as resampling methods, are based on drawing random samples from the given sample to make inferences about the population. An example of this is the bootstrap methods, which is useful to calculate confidence intervals.

4.5.1 Bootstrapping

Example